RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Génie Logiciel pour les scientifiques et les ingénieur.e.s météo

cours 1 : Frank Guibert DSM/LabIA - frank.guibert@meteo.fr
cours 2: Théo Tournier DSM/LabIA: theo.tournier@meteo.fr
cours 3: Oscar Dewasmes DSM/LabIA: oscar.dewasmes@meteo.fr

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Première partie, sommaire

1. **Importance du génie logiciel en entreprise**
2. **Cas particulier de Météo-France**
3. **La dette technique**
4. **Les paradigmes de développement**
5. **Les bonnes pratiques illustrées**
6. **Le cycle de vie d'un projet logiciel**
7. **Les méthodologies de développement logiciel**

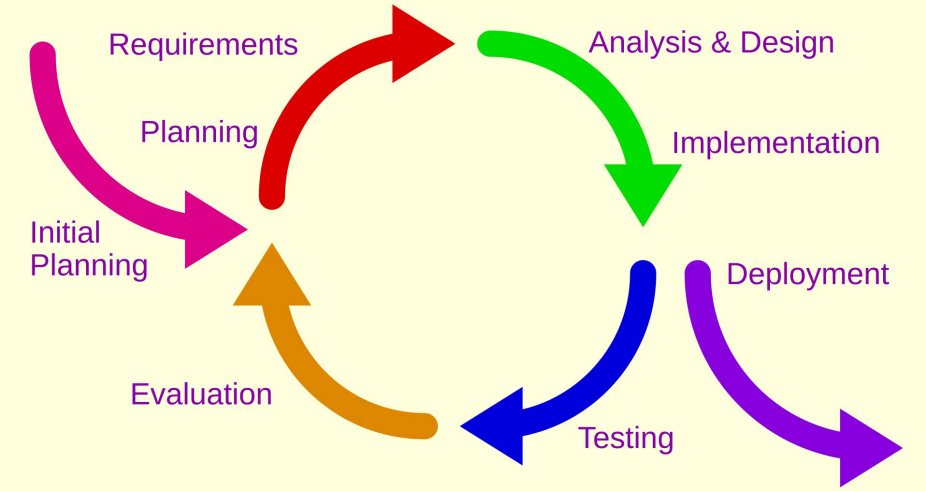
RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Selon vous, comment définir le génie logiciel ?

Une vaste définition

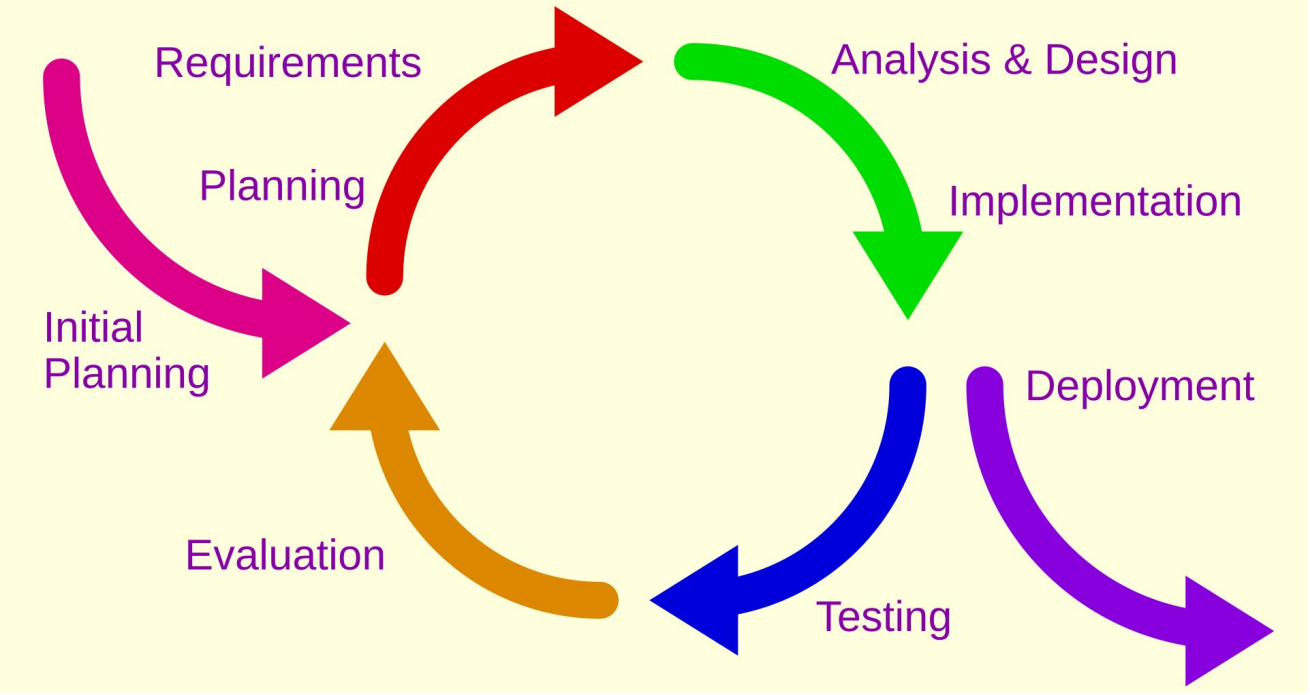
Toutes les compétences techniques, créatives et méthodologiques pour concevoir, développer, tester et maintenir des logiciels.



→ Le Génie Logiciel est donc un “mal nécessaire” et peut même devenir un “atout” pour faire + de physique, + de météo, + vite et de façon collaborative.

Définition traditionnelle du génie logiciel

« L'ensemble des activités de conception et de mise en œuvre des produits et des procédures tendant à rationaliser la production du logiciel et son suivi ».



RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Dans ce cours : une approche pragmatique

- **Le Génie Logiciel “classique” s’adresse plus à des profils d’ingénieur.e.s en informatique**
- **Nous allons donc nous orienter vers des choses plus simples, plus concrètes, issues de nos retours d’expériences à Météo-France, des “bonnes pratiques”**
- **Afin que vous ayez des outils concrets pour vos projets métiers, vos thèses et vos développements techniques**
- **Une partie des vidéos et liens proposés sont en anglais, pour vous “pousser” vers le web anglophone plus riche sur le sujet.**

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Objectifs du cours

- Vous **convaincre de l'intérêt du Génie Logiciel** pour vous faciliter la vie dans vos carrières d'ingénieur.e.s à Météo-France, sur des postes variés, scientifiques et/ou techniques.

- **Vous partager des connaissances**, des savoir-faire et des réflexes applicables sur le terrain pour vous permettre de faire **+ de sciences**, **+ de physique**, **+ de météo** et **- de code** ou **- de maintenance de code**.

- Vous habituez à travailler en Anglais sur ces sujets

- **Vous pouvez utiliser ce support comme boîte à outils lors de vos développements.**

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Un peu de statistiques

- **70% of software projects in 2024 either failed or faced substantial difficulties**

- On average, large IT projects exceed their budgets by 45% and are completed 7% later than planned, while delivering only 56% of their projected value.

- 20% of failures attributed to technical challenges.

- In 2024, nearly 40% of failed projects mentioned unclear or changing requirements as a primary reason for failure.

Source: Project Management Institute

Pourquoi est-ce important à Météo-France et ailleurs ?

Constats :

- **Faire des systèmes/logiciels coûte cher et échoue souvent**

- **Météo-France est un Opérateur d'Importance Vitale (OIV)** dédié à la protection des personnes et des biens. Donc certains systèmes sont des **SIV (Systèmes d'Importance Vitale).**

- **Tous les produits / service vendus ne doivent pas coûter plus cher que ce qu'ils rapportent sur le long terme.**

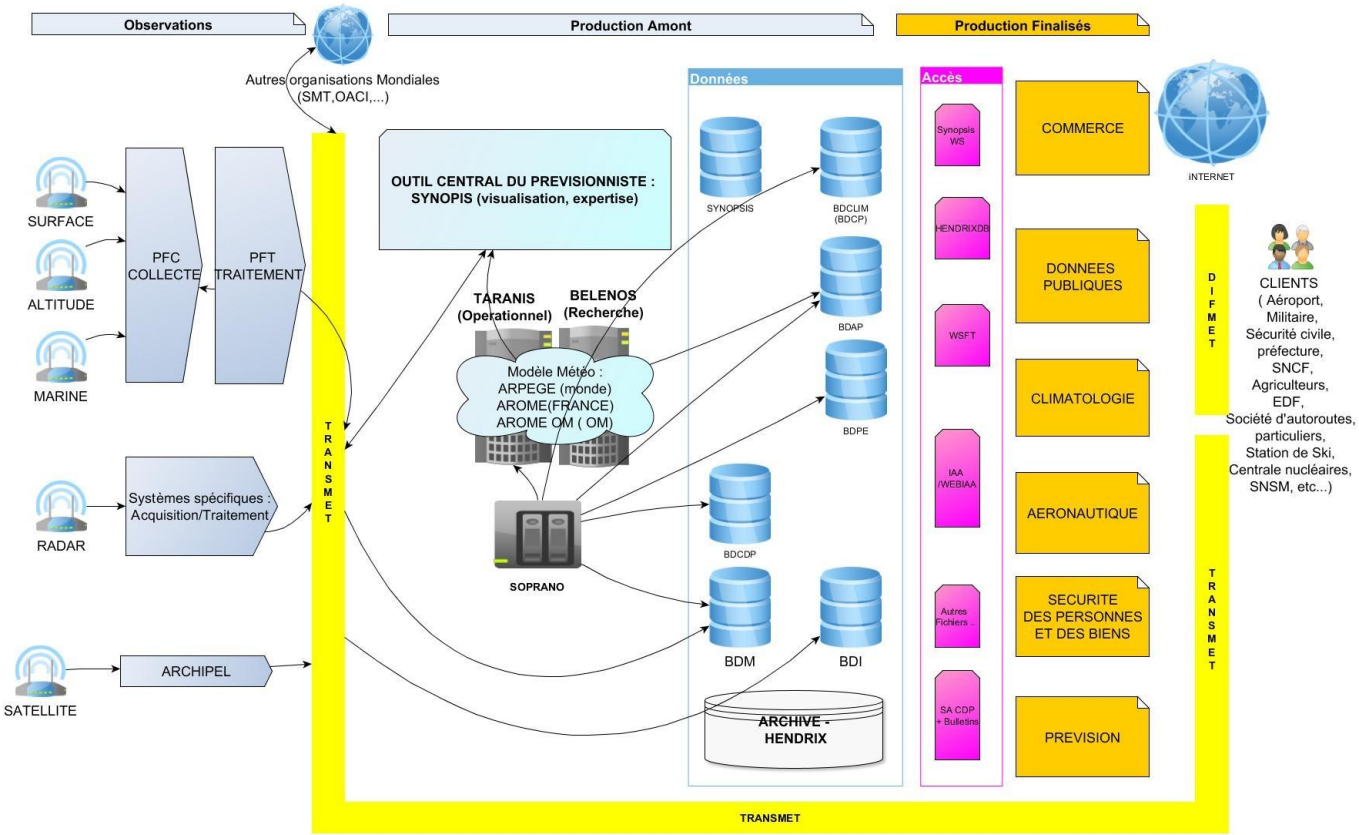
"Il FAUT impérativement que ça fonctionne" et on sait grâce aux statistiques et à nos retours d'expériences que **"c'est compliqué".**

Les logiciels et le code à Météo-France

On peut voir Météo-France (et tout centre Météo National) comme une organisation qui s'appuie sur un **ensemble de systèmes informatiques interconnectés**:

- Modèles de PNT et de Climat (et maintenant modèles d'IA)
- Chaînes de production finalisées
- Outils du prévisionniste
- Outils aéronautiques
- Outils de transmission de données météos
- Sites web : grand public, vigilance
- Application mobile

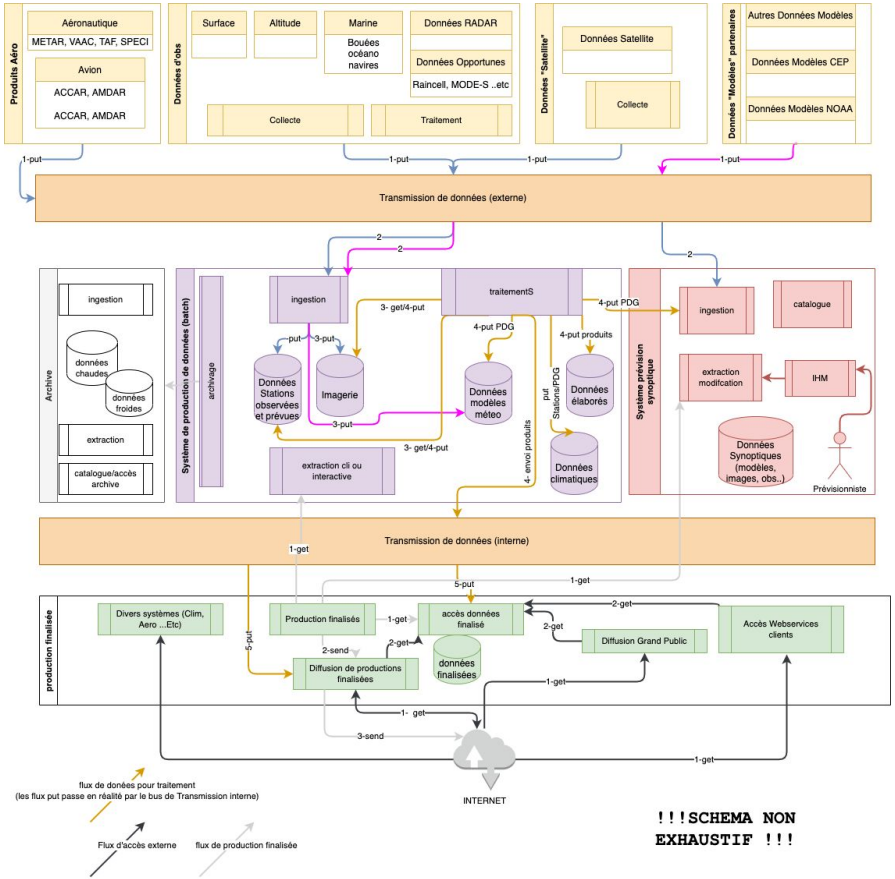
Le système d'information de Météo-France



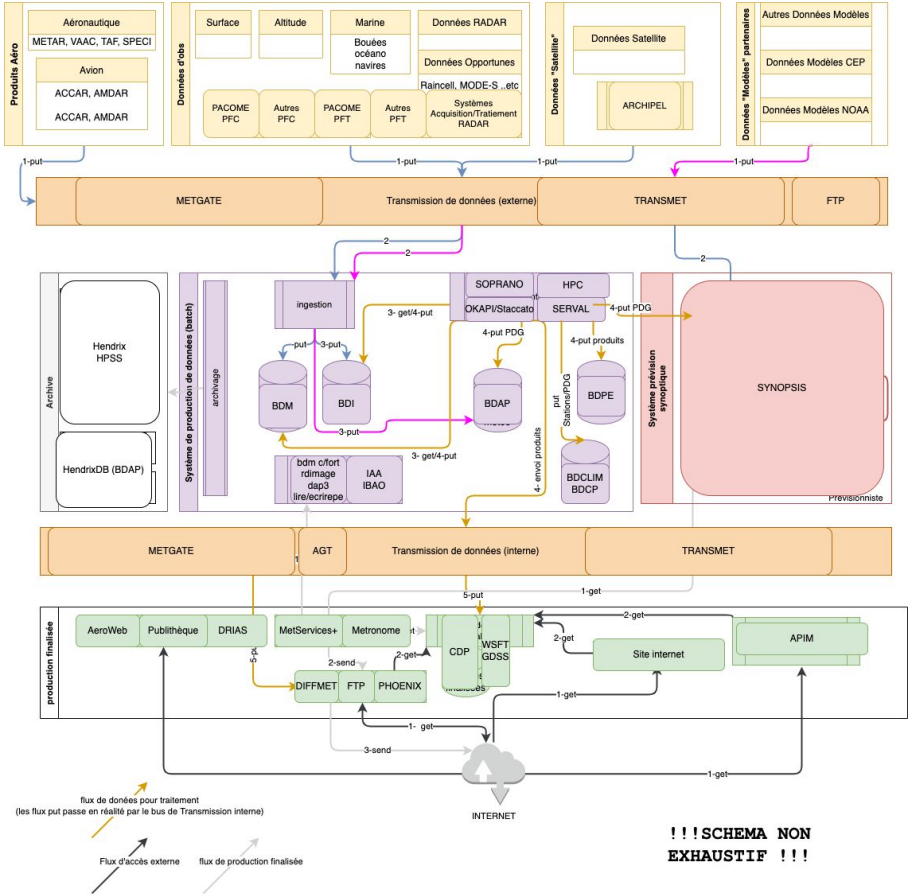
RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Architecture fonctionnelle du SI



Architecture fonctionnelle du SI: mapping systèmes



1 boîte sur les schémas = 1 gros logiciel informatique ou “système”

Le CNRM

- UMR CNRS/Météo-France ~ 400 personnels ~ 30 équipes
- Activités de développement logiciel variées
  - Simulation numérique, analyse de données, visualisation, outils de gestion de données, outils de compilation.
  - Python, Fortran, Perl, C, C++, R, bash
  - De notebooks jupyter aux dév. des modèles de prévision numérique
- Le développement logiciel n'est pas structuré ni encadré
  - Pratiques de développement datées en 2025
  - Dette technique conséquente
  - Charge de travail croissante pour les agent.es : "moins de science".

Comment expliquer un tel retard?

- Manque de connaissances et **compétences**
  - Personnel scientifique pas/peu formé au dev. logiciel
- Manque de **motivation**
  - Personnel intrinsèquement motivé par la réponse à des questions scientifiques
  - Effort de dev. logiciel à rebours de la réalité de l'évaluation de la recherche
  - Absence structurelle de valorisation de la qualité logicielle
- Manque d'**infrastructure**
  - Pas de socle commun encadrant les pratiques et technologies
  - Peu de personnel expert en ingénierie logicielle, éparpillé et non coordonné
  - Financements par projets ne couvrant pas le développement et/ou la maintenance

Éléments de solution provenant d'autres institutions

(e.g. DLR, MetOffice, ECMWF, Alan Turing Institute, médialab Sciences Po, *INRIA?*)

- **Homogénéiser les pratiques** et fournir un socle commun sur lequel s'appuyer (*pile technique*)
  - Ciblage la documentation et les besoins de formation
  - Veille technique coordonnée et mutualisée
  - Échanges inter-équipes facilités, émulation
- **Construire un pôle d'expertise** en ingénierie logicielle
  - Fournir un accompagnement expert pour le développement et le transfert à l'opérationnel
  - Animer et faciliter l'échange de connaissances
- **Valoriser le logiciel** comme production scientifique de premier plan
  - Chiffrage du développement et de la maintenance dans les demandes de financement
  - Encourager la publication des logiciels (JOSS, SoftwareX)
  - Faire entrer la production logicielle dans l'évaluation du laboratoire (HCERES)

Cas particulier du modèle de prévision numérique

Une base de code à travers les âges:

- ~ 1990 FORTRAN 77 + extensions Cray (10aine de dev.)
- ~ 1996 Passage à Fortran 90
- ~ 2000 Parallélisme à mémoire distribuée (MPI)
- ~ 2022 Calcul sur accélérateurs (GPGPU)

En 2025:

- Base de code massive, multi-langages (C, C++, Fortran 90)
- Plusieurs centaines de contributeur.ices
- Code partagé entre plusieurs institutions aux intérêts parfois différents.
- Peu, voire pas de documentation écrite.
- Base de code très difficile à manipuler, source d'erreurs.
- Faible degré de modularité, pas de tests unitaires.

Qui écrit du code à Météo-France ?

- L’écriture de code sur des projets collectifs concerne de nombreux agent.e.s:
  - Les chercheurs.euses de la DESR
  - Les développeurs.euses de la DSI
  - Les agents de la DSM, de la DCSC
  - Les agents de la DSO
  - Les chargés d’études de la DSM
  - Les agents de la DCSC

→ Il est donc **très probable que certain.e.s d’entre vous passent une part significative de leur temps à “écrire du code”**, qu’il soit scientifique ou plus technique.

→ Dans le cas contraire, vous serez peut-être des chef.f.e.s de projets, des “clients internes” et donc le code vous concerne aussi.

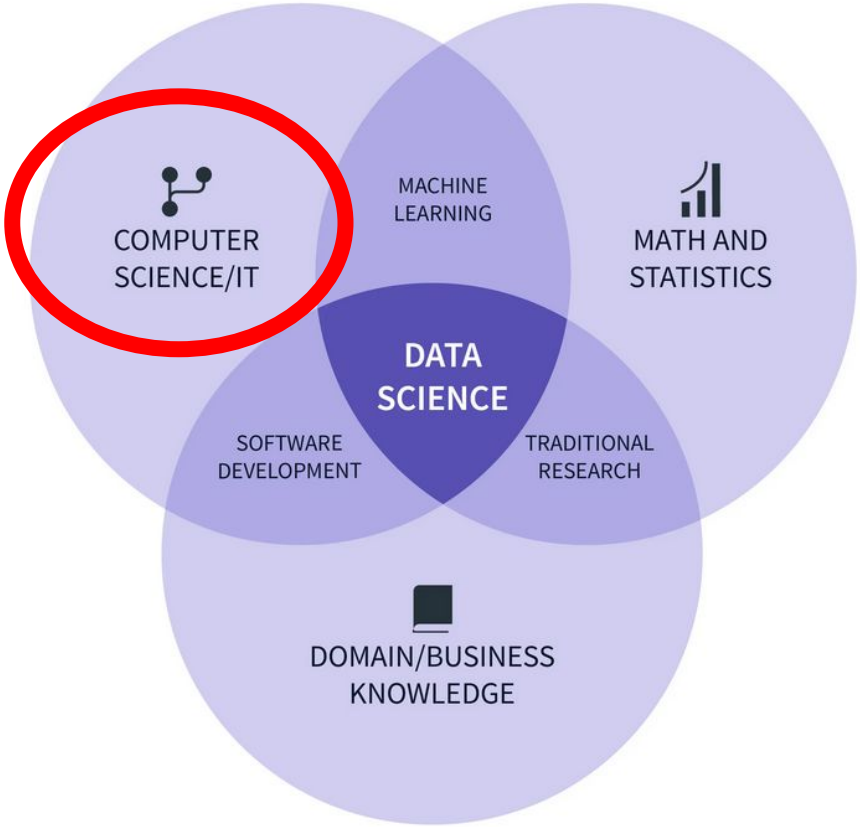
Le pourquoi du génie logiciel : un détour par l'IA

L'intelligence artificielle bouscule de nombreux secteurs depuis ~ 10 ans :

- Modèles de langage (ChatGPT, Gemini…)
- Modèles de traitement d'images
- Modèles de Prévision Numérique du temps (AIFS, PanguWeather…)
- Modèles de génération de bulletins météo Marine (projet MF)

→ L'IA requiert de solide compétences en informatique et c'est souvent le "parent pauvre" des formations scientifiques

Le pourquoi du génie logiciel : l'IA/Data Science



La dette technique / Technical Debt

« **Sortir une première itération de code, c'est comme s'endetter**. Une petite dette accélère le développement tant qu'elle est remboursée rapidement par une réécriture... Le danger survient lorsque la dette n'est pas remboursée. **Chaque minute passée sur un code pas tout à fait correct compte comme un intérêt sur cette dette**. Des organisations entières peuvent être paralysées sous la charge de la dette d'une implémentation non consolidée, qu'elle soit orientée objet ou autre[3]... »

Ward Cunningham, 1992

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

La dette technique

Dette financière	Dette technique
Mensualités = Capital + intérêts	Capital = Architecture / code de mauvaise qualité Intérêts: les erreurs à corriger
Plus le capital est important, ou moins le capital est remboursé, plus les intérêts sont élevés	Plus la qualité du code est mauvaise, et plus nous étendons ce code, plus l'effort pour corriger les erreurs sera important

Eric Higgins : *The technical debt is like Tetris: you can't win. You can only control how quickly you lose*

La dette technique

La dette technique peut être considérée comme

- « **visible** » lorsqu'elle regroupe l'ensemble des défauts qui ont des effets visibles par les utilisateurs

- « **invisible** », si elle regroupe les défauts, qui ne sont pas visibles par les utilisateurs (mauvaise qualité du code, ou complexité structurelle de la plateforme).

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Les causes de la dette technique

- Budget limité et contraintes temporelles fortes
- Manque de compétences sur les sujets traités
- Sélection de profils qui ne sont pas adaptés aux besoins
- sous-estimation de la complexité du sujet => architecture simpliste
- over-engineering => design trop complexe
- pas de revue de code
- insuffisance des tests

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Le quadrant de Martin Fowler

Imprudente / excessive

Dette délibérée / intentionnelle	Dette involontaire / non intentionnelle
Pression économique, ou pression sur les délais	Ignorance des bonnes pratiques (même les plus basiques)
Décision de faire du quick&dirty en raison des contraintes de temps.	Pas de gestion de la qualité (ou gestion défaillante)
Pas le temps de faire le design	Sous-estimation des tests
Ou choisir la bonne technologie	Processus de développement non structurés
	Documentation négligée
	Over-engineering (architecture ou code trop complexe)

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Les différents types de dette technique

- **Architecture**
  - Sous estimation de la complexité
  - Non compréhension des besoins
  - Manque de connaissance des architectures logicielles
  - Manque de temps (pas de Proof Of Concept)

- **Code**
  - pas de revue de code
  - non respect des patterns et bonnes pratiques

Les différents types de dette technique

- **Infrastructure (environnement OPER)**
  - difficultés de mise à jour
  - manque de monitoring
  - manque de préparation des équipes OPER
  - plateforme OPER vétuste
  - manque de préparation de la phase OPER

- **Tests**
  - pas assez de tests (unitaires, fonctionnels, de charge)
  - pas d'automatisation des tests
  - manque de suivi de tests et/ou de corrections

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

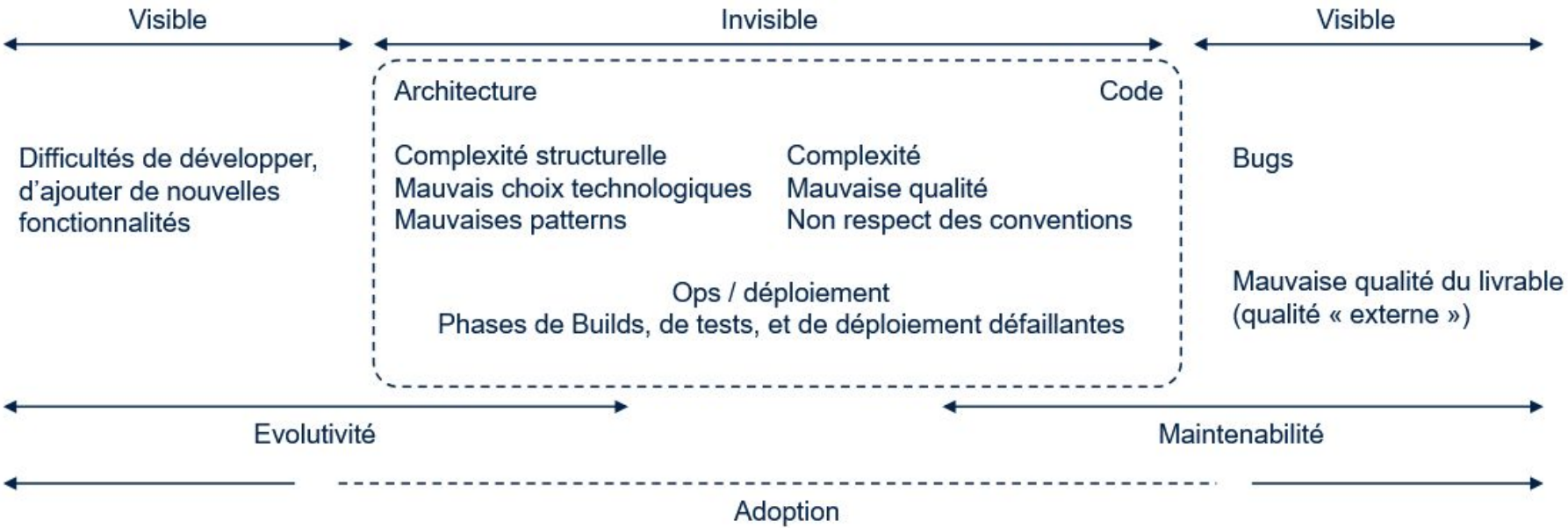
Les problèmes posés par la dette technique

Augmentation des coûts de :

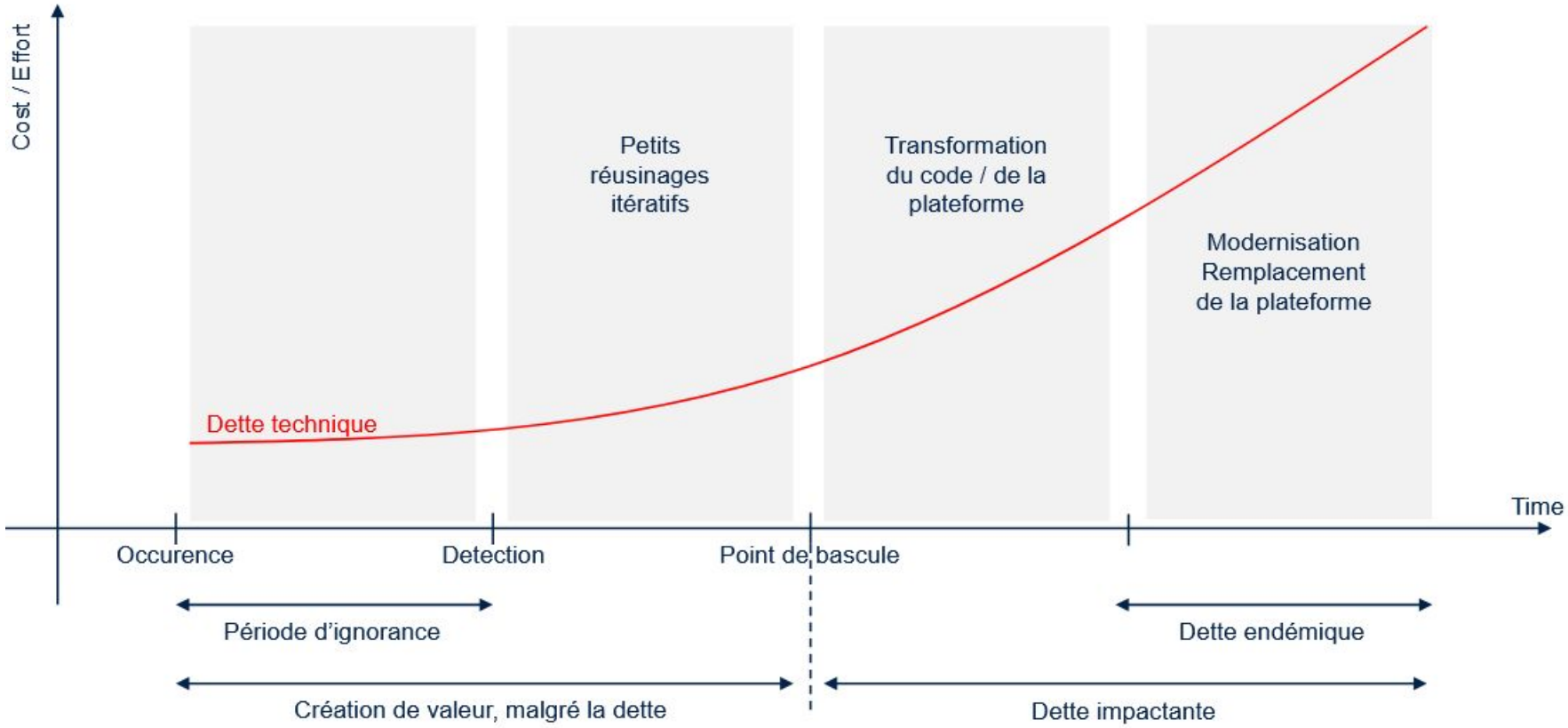
- **Maintenabilité:** correction de bugs, adaptation à de nouvelles technologies.
- **Évolutivité:** ajouts de nouvelles fonctionnalités longs et fastidieux.

Risque de <u>non adoption</u>: une application endettée et couteuse à maintenir et faire évoluer peut entrainer l'adoption de solutions alternatives.

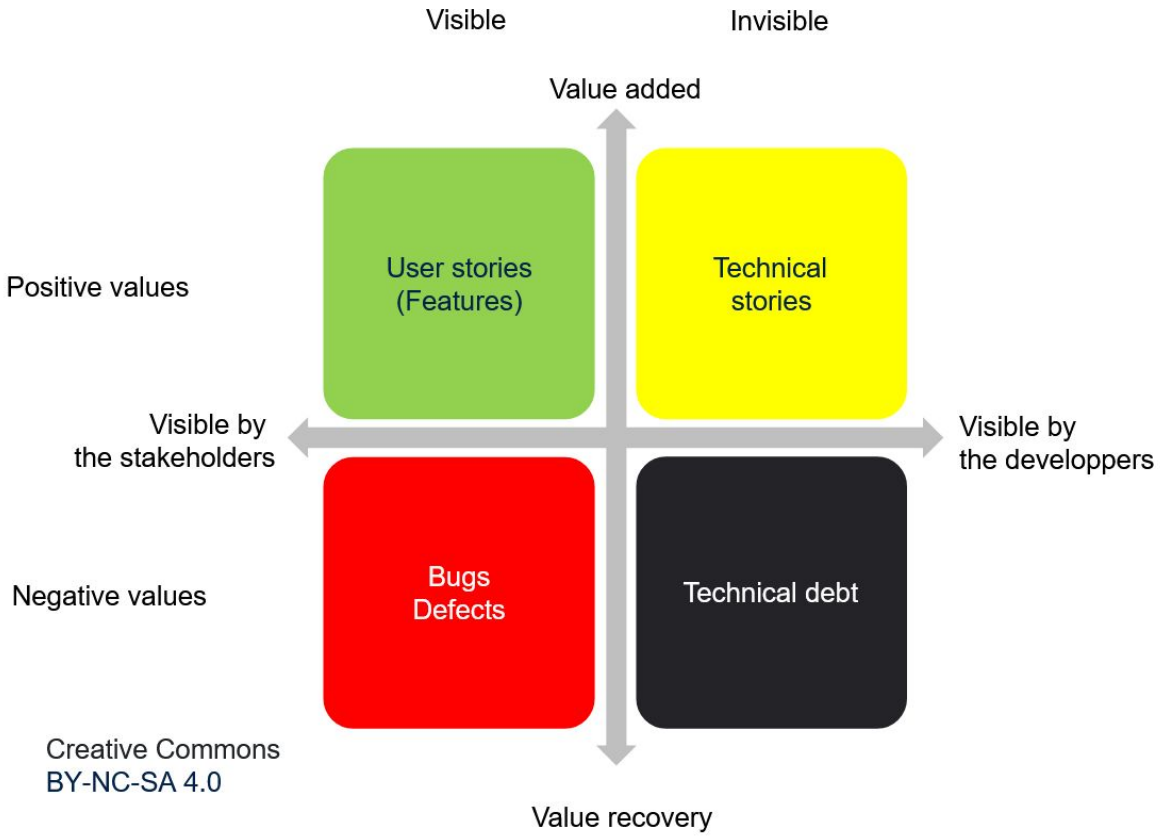
La dette technique: causes et conséquences



La dette technique: stratégie de réduction

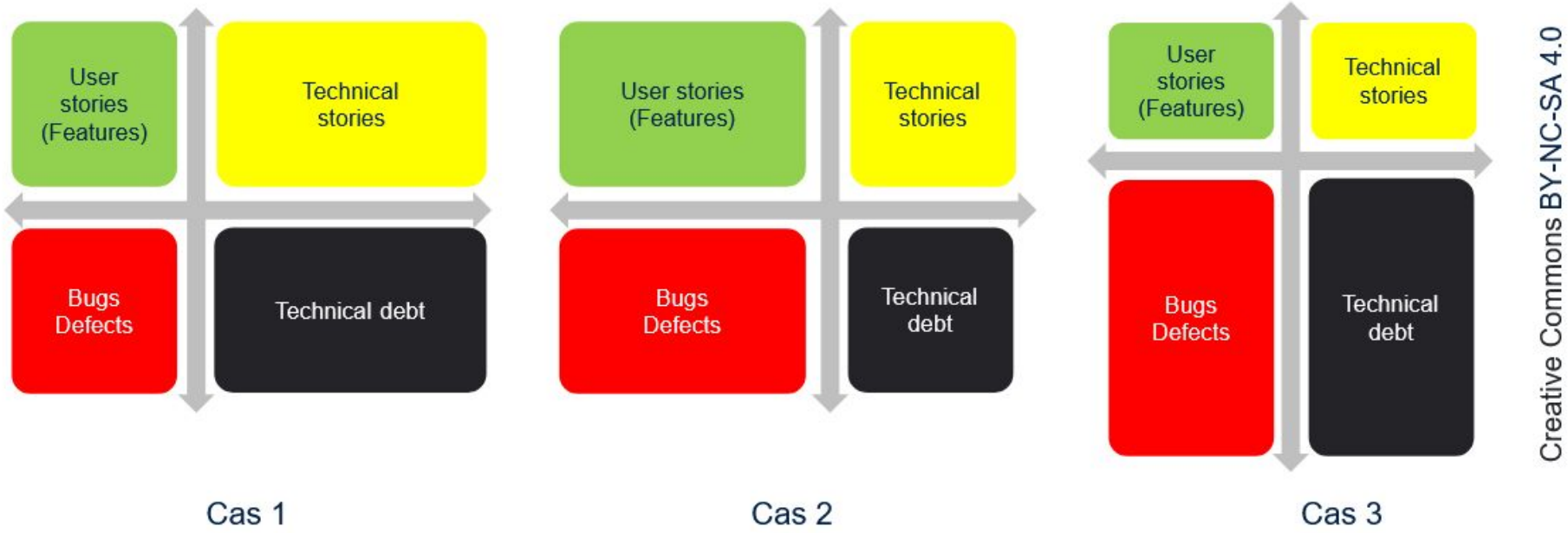


La dette technique: ajout de tickets dédiés



Creative Commons
BY-NC-SA 4.0

La dette technique: Equilibrage du backlog



Creative Commons BY-NC-SA 4.0

Figure 4: Equilibrage du Backlog

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Contrôle de la dette

- **Inclure la dette dans le projet**
  - Qualité du code et des architectures aussi importantes que les fonctionnalités utilisateur
  - Sensibiliser les parties prenantes aux risques induits par des choix court termistes

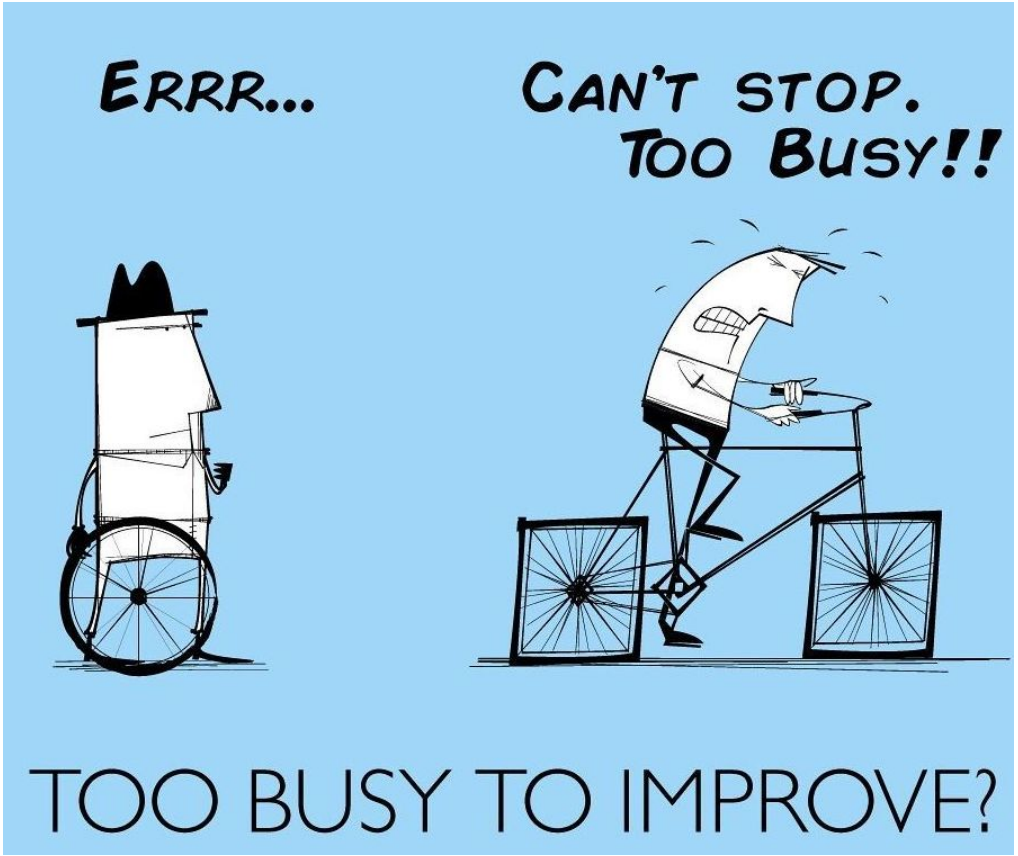
- **Prêter attention aux profils recrutés**: dev, tech lead, data scientists

- **Sensibiliser les équipes projet**
- **Revues d'architecture et de code**
- **Automatisation des processus et des tests**

Conclusion sur la dette technique

- **Qualité: “comment on fait” est aussi important que “qu’est ce qu’on fait” au niveau des livrables**
- **Invite à une stratégie sur le long terme**
- **Oblige l’ensemble de l’équipe à acquérir plus de compétences sur les livrables du projet**

Conclusion sur la dette technique



Alan O'Rourke (CC-BY 2.0)

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Les paradigmes de travail



RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Les paradigmes de développement

- De quoi parle t'on ?
  - Des principes de travail applicables pour vos projets avec du code: logiciels DSI, thèses, projets métiers, …
  - Des principes éprouvés sur le terrain, certains depuis plusieurs décennies dans l'industrie
  - Des retours issus des travaux à Météo-France
  - Des acronymes anglophones mnémotechniques

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

KISS: Keep It Simple Stupid

- Introduit par Kelly Johnson dans l'industrie aéronautique
- Faire au plus simple
- Ne pas créer de framework maison
- Ne pas éclater le code sur de nombreux dépôts interdépendant
- exemple à ne pas reproduire : metwork
- Similaire au rasoir d'Ockham : « les hypothèses suffisantes les plus simples doivent être préférées (il faut et il suffit) »
- Philosophie d'Unix: "Écrivez des programmes qui font une seule chose et la font bien"

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

KISS: Keep It Simple Stupid : comment faire ?

- **Ecrire des programmes courts:** Des fonctions et des classes concises qui traitent un problème à la fois.
- **Supprimer le code non utilisé:** Permet de réduire la complexité du projet et le temps de prise en main
- **Se concentrer sur la lisibilité:** Ecrire du code transparent et (presque) évident à comprendre pour les autres
- **Composer des briques simples:** Utiliser le code existant en combinant des petites fonctions plutôt que de ré-écrire des fonctionnalités
- **Programmation modulaire:** Découper l'application en modules (module Python = 1 fichier .py) indépendants. Cela permet d'organiser le code et améliorer la flexibilité et la réutilisabilité.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

KISS: Keep It Simple Stupid: un contre exemple

```python
def convert_temperature(temperature, from_scale, to_scale):
    if from_scale == 'F' and to_scale == 'C':
        return (temperature - 32) * 5 / 9  # Fahrenheit to Celsius
    elif from_scale == 'C' and to_scale == 'F':
        return (temperature * 9 / 5) + 32  # Celsius to Fahrenheit
    elif from_scale == 'F' and to_scale == 'K':
        return (temperature - 32) * 5 / 9 + 273.15  # Fahrenheit to Kelvin
    elif from_scale == 'K' and to_scale == 'F':
        return (temperature - 273.15) * 9 / 5 + 32  # Kelvin to Fahrenheit
    elif from_scale == 'C' and to_scale == 'K':
        return temperature + 273.15  # Celsius to Kelvin
    elif from_scale == 'K' and to_scale == 'C':
        return temperature - 273.15  # Kelvin to Celsius
    return temperature
```

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

KISS: Keep It Simple Stupid: des fonctions simples

```
def fahrenheit_to_celsius(temp):
    return (temp - 32) * 5 / 9

def celsius_to_fahrenheit(temp):
    return (temp * 9 / 5) + 32

def fahrenheit_to_kelvin(temp):
    return (temp - 32) * 5 / 9 + 273.15

def kelvin_to_fahrenheit(temp):
    return (temp - 273.15) * 9 / 5 + 32

def celsius_to_kelvin(temp):
    return temp + 273.15

def kelvin_to_celsius(temp):
    return temp - 273.15
```

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

KISS: Keep It Simple Stupid: une table d'association

```
conversion_functions = {
    ('F', 'C'): fahrenheit_to_celsius,
    ('C', 'F'): celsius_to_fahrenheit,
    ('F', 'K'): fahrenheit_to_kelvin,
    ('K', 'F'): kelvin_to_fahrenheit,
    ('C', 'K'): celsius_to_kelvin,
    ('K', 'C'): kelvin_to_celsius
}

def convert_temperature(temperature, from_scale, to_scale):
    if from_scale == to_scale:
        return temperature
    convert_fn = conversion_functions.get((from_scale, to_scale))
    return convert_fn(temperature) if convert_fn else temperature
```

DRY: Don’t Repeat yourself - “Duplication is Evil”

« Dans un système, toute connaissance doit avoir une représentation unique, non ambiguë, faisant autorité ». *The Pragmatic Programmer par* Andy Hunt et Dave Thomas

- Essayer de ne pas copier-coller de code en premier lieu
- Identifier les duplications dans votre code
- Faire des fonctions et des classes ré-utilisables
- Utilisez des modules et des librairies Python sur étagère (https://pypi.org/)
- Les packages Python les plus plus utilisés: https://hugovk.github.io/top-pypi-packages/

1.	boto3	1 438 455 634
2.	urllib3	1 106 102 051
3.	botocore	1 010 375 477
4.	typing-extensions	990 302 186
5.	requests	959 087 169
6.	certifi	955 309 470

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

DRY: Don’t Repeat yourself: exemple en Python

```python
student1_name = 'Alice'
student1_age = 20
student1_major = 'Mathematics'

student2_name = 'Bob'
student2_age = 22
student2_major = 'Physics'
```



```python
# Using dictionaries to store student data
students = {
    'student1': {'name': 'Alice', 'age': 20, 'major': 'Mathematics'},
    'student2': {'name': 'Bob', 'age': 22, 'major': 'Physics'}
}

print(students['student1']['name'])  # Output: Alice
print(students['student2']['age'])   # Output: 22
```

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

YAGNI: You Aren't Gonna Need It

- Principe introduit par Ron Jeffries dans le cadre de l'eXtreme Programming (XP)
- Minimiser le temps CPU et le temps mental des développeurs et développeuses
- N'écrire que ce dont on a besoin
- Ne pas faire d'anticipation de besoins futurs non formalisés
- Ecrire le moins de code possible
- Implémenter le moins de fonctionnalité possible pour satisfaire les besoins du projet/client.e
- Parfois ne pas coder DU TOUT !

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Don’t Reinvent The Wheel

- **Ne pas recoder des choses qui existent déjà**
- **Connaître et utiliser les librairies standards des langages**
- Utiliser des librairies open-source
- Chercher sur pypi et github
- Utiliser des librairies internes à Météo-France
- Utiliser des librairies des partenaires (ECMWF)
- Contribuer à des librairies existantes
- Créer si besoin des librairies ou projets communautaires

Pour faire ce support j’ai appliqué ce principe au maximum en utilisant des contenus communautaires déjà faits !

Quelques conseils supplémentaires

- **Écrivez des programmes pour les humains, pas pour les ordinateurs:** codez proprement, structurez vos algorithmes, commentez votre code, utilisez des noms de variable qui ont un sens, soignez le style et le formatage, etc.

- *Code is read far more often than it is written.* Ne croyez pas que vous ne relirez jamais votre code (ou même que personne n'aura jamais à le lire), ou que vous aurez le temps de le refaire mieux plus tard…

- **« *Premature optimization is the root of all evil.* »** (Donald Knuth) Mieux vaut un code lent mais juste et maintenable qu'un code rapide et faux ou incompréhensible. Dans l'ordre absolu des priorités:
  - *Make it work.*
  - *Make it right.*
  - *Make it fast.*

L’art du nommage

- “Il n'y a que deux choses difficiles en informatique : **l'invalidation du cache et le f<u>ait de nommer les choses</u>**.”

- **Don’t Make me think, give Meaningful names**

    - Pour un système d’archivage de données:
        - Exemple explicite: “archiver”
        - Contre exemple: “Hendrix” (MF)

    - Pour un logiciel destiné aux prévisionnistes:
        - Exemple explicite: “forecast”, “previ”
        - Contre exemple moyen explicite: “Synopsis” (MF)

Parfois la culture interne favorise les nommages rigolos…mais pas explicite !

Le piège de l’égo

[The Ten commandments of egoless programming]

Understand and accept that you will make mistakes. The point is to find them early, before they make it into production. Fortunately, except for the few of us developing rocket guidance software at JPL, mistakes are rarely fatal in our industry, so we can, and should, learn, laugh, and move on.

You are not your code. Remember that the entire point of a review is to find problems, and problems will be found. Don't take it personally when one is uncovered.

No matter how much "karate" you know, someone else will always know more. Such an individual can teach you some new moves if you ask. Seek and accept input from others, *especially* when you think it's not needed.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Le piège de l’égo

Don't rewrite code without consultation. There's a fine line between "fixing code" and "rewriting code." Know the difference, and pursue stylistic changes within the framework of a code review, not as a lone enforcer.

Treat people who know less than you with respect, deference, and patience. Nontechnical people who deal with developers on a regular basis almost universally hold the opinion that we are prima donnas at best and crybabies at worst. Don't reinforce this stereotype with anger and impatience.

The only constant in the world is change. Be open to it and accept it with a smile. Look at each change to your requirements, platform, or tool as a new challenge, not as some serious inconvenience to be fought.

The only true authority stems from knowledge, not from position. Knowledge engenders authority, and authority engenders respect – so if you want respect in an egoless environment, cultivate knowledge.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Le piège de l’égo

Fight for what you believe, but gracefully accept defeat. Understand that sometimes your ideas will be overruled. Even if you do turn out to be right, don't take revenge or say, "I told you so" more than a few times at most, and don't make your dearly departed idea a martyr or rallying cry.

Don't be "the guy in the room." Don't be the guy coding in the dark office emerging only to buy cola. The guy in the room is out of touch, out of sight, and out of control and has no place in an open, collaborative environment.

Critique code instead of people – be kind to the coder, not to the code. As much as possible, make all of your comments positive and oriented to improving the code. Relate comments to local standards, program specs, increased performance, etc.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

De l’usage modéré et réfléchi des paradigmes

- **Ces principes sont parfois antagonistes (DRY versus KISS)**
- **Ces paradigmes ne sont pas des règles rigides mais des pratiques à composer intelligemment et en contexte**
- **Votre code ne sera jamais parfait, acceptez le et faites des compromis**

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Rester à la page/ Se former

- S'abonner à des newsletters Python (PyCoders weekly)
- Lire des blogs techniques (RealPython)
- Lire les release notes des nouvelles versions des langages que vous utilisez
- Regarder des vidéos youtube
- Suivre des formations internes ou externes
- Lire des papiers de recherche (en IA)
- Parcourir les codes open-source des librairies les plus utilisées de vos langages (numpy, scipy, pytorch, …)

Cela représente environ ~ 10 % de notre temps à DSM/LabIA

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Fin de la première partie

Deuxième partie, sommaire

1. Importance du génie logiciel en entreprise
2. Cas particulier de Météo-France
3. La dette technique
4. Les paradigmes de développement
5. **Les bonnes pratiques illustrées**
6. Le cycle de vie d'un projet logiciel
7. Les méthodologies de développement logiciel

test.py 4, U

test.py > ...

```python
import numpy as np
import torch
from fire.data import get_xarray #corplot
from fire.settings import DSParisHyperParameters

hparams = DSParisHyperParameters()

def get_dataloader():

    maxi_feu, min_feu, d_FINAL_X, d_GFED = get_xarray()

    d_FINAL_X_numpy = d_FINAL_X.values
    d_FINAL_X_numpy = np.float32(d_FINAL_X_numpy)

    d_GFED = d_GFED.to_dataarray() #IMPORTING GFED
    d_GFED_numpy = d_GFED.to_numpy()

    d_FINAL_X_numpy = torch.cat((d_GFED_numpy, d_FINAL_X_numpy), 3) #[time,720,1440,(1 + feature]) #Lourd

    train_loader = torch.utils.data.DataLoader(d_FINAL_X_numpy[:1636352], batch_size=hparams.batch_size)
    valid_loader = torch.utils.data.DataLoader(d_FINAL_X_numpy[1636352:1780736], batch_size=hparams.batch_size)

    ##return train_loader, valid_loader, var_feu, mean_feu
    return train_loader, valid_loader, maxi_feu, min_feu

```

Exemple 1:

4 mauvaises pratiques à trouver

Correction Exemple 1

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Documentation

Décrire ce que fait le code

En théorie :

- La documentation répond à un besoin :

« Regrouper et ordonner les informations d'un projet pour que tous les membres de l'équipe puissent facilement commencer à contribuer au code. »

En pratique :

- La doc est en grande partie une reformulation des besoins fonctionnels.
- Une partie de la doc se trouve dans le code sous forme de docstrings.
- La documentation est utile à tous les membres de l'équipe, pas seulement aux nouveaux arrivants.
- La documentation est versionnée

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Lisibilité

Écrire pour le lecteur

En théorie :

- Le code doit être écrit pour l'ordinateur ET le lecteur humain.
  - L'exécution du code par l'ordinateur doit donner le comportement attendu.
  - La lecture du code doit convaincre le lecteur que son exécution donnera le comportement attendu.

En pratique :

- Écrire son code pour la prochaine personne qui le lira. Que la lecture soit facile, fluide et convaincante.
- 80% du temps, vous serez le prochain lecteur de votre propre code.

Retour Exemple 1

Lisibilité

Le Nommage

En théorie :

- Nommer vos modules, classes, fonctions, et variables de façon à ce que leur responsabilité ou ce qu'elles représentent soit le plus explicite possible.
- Respecter le standard de nommage python.

⚠️ **Nom de projets**

En pratique :

- Éviter les abréviations dans les noms.
- Éviter les noms de variables ne contenant qu'une lettre.
- Ne pas hésiter à utiliser des longs noms.
- Utiliser le *snake_case* pour les fichiers, fonctions et variables, *CamelCase* pour les classes, *kebab-case* pour les noms de projet.

Retour Exemple 1

Lisibilité

KISS: Keep It Simple Stupid

En théorie :

- Faire au plus simple
- Complexe != Compliqué
  On peut créer un système complexe simplement
- Philosophie Unix
  « Écrivez des programmes qui font une seule chose et la font bien »

En pratique :

- Une fonction / classe / module ne fait qu'une chose, et bien.
- Ne pas éclater le code sur de nombreux dépôts interdépendants

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Lisibilité

Les Magic Numbers

En théorie :

- Bannir l'usage de Magic Numbers.
- Entraîne la violation de plusieurs paradigmes: DRY, pas de nommage, pas de documentation.

En pratique :

- Définir des variables ou constantes qui seront utilisées tout au long du code plutôt que de dupliquer le même nombre.
- Ne jamais modifier la valeur d'une constante.
- Implémenter les calculs qui mènent au résultats plutôt que le résultat lui même.
- Ne pas hésiter à utiliser des longs noms pour que le nommage soit explicite.

Correction Exemple 1

```python
import numpy as np
import torch
from fire.data import get_xarray #corplot
from fire.settings import DSParisHyperParameters

hparams = DSParisHyperParameters()

def get_dataloaders():

    maxi_feu, min_feu, d_FINAL_X, d_GFED = get_xarray()

    d_FINAL_X_numpy = d_FINAL_X.values
    d_FINAL_X_numpy = np.float32(d_FINAL_X_numpy)

    d_GFED = d_GFED.to_dataarray() #IMPORTING GFED
    d_GFED_numpy = d_GFED.to_numpy()

    #[time,720,1440,(1 + feature]) #Lourd
    d_FINAL_X_numpy = torch.cat((d_GFED_numpy, d_FINAL_X_numpy), 3)

    train_loader = torch.utils.data.DataLoader(
        d_FINAL_X_numpy[:1636352], batch_size=hparams.batch_size
        )
    valid_loader = torch.utils.data.DataLoader(
        d_FINAL_X_numpy[1636352:1780736], batch_size=hparams.batch_size
        )

    ##return train_loader, valid_loader, var_feu, mean_feu
    return train_loader, valid_loader, maxi_feu, min_feu
```

```python
import numpy as np
import torch
from torch.utils.data import DataLoader
from fire.data import get_xarray

def get_dataloaders(batch_size, pct_in_train):
    """Instanciate train and valid dataloader from:
    - climate fields: temperature, reative humidity, precipitations and wind (10m).
    - Global Fire Emission Database (GFED).

    Args:
        batch_size: Size of the batch to use in the dataloader.
        pct_in_train: Percentage of the whole dataset to use in train.
    """
    climate_dataset, fire_emissions_dataset = get_xarray()  # Load xarray datasets

    # Convert xarray.dataset to numpy array
    climate_values = np.float32(climate_dataset.values)  # concvert to float 32
    fire_emission_values = fire_emissions_dataset.values

    # Concatenate climate and fire dataset
    dataset = torch.cat((fire_emission_values, climate_values))

    # Split dataset into train (0 -> pct_in_train) and validation (pct_in_train -> end)
    end_train_idx = int(len(dataset) * pct_in_train)
    train_dataset = dataset[:end_train_idx]
    valid_dataset = dataset[end_train_idx:]

    # Create train and valid DataLoaders
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size)
    valid_dataloader = DataLoader(valid_dataset, batch_size=batch_size)

    return train_dataloader, valid_dataloader
```

mean_and_sum.py U

mean_and_sum.py > ...

```python
def compute_sum(liste):
    """Calcule la somme d'une liste.
    Args:
        liste: liste de nombres.
    Returns:
        la somme de la liste.
    """
    total = 0
    for element in liste:
        total += element
    return total

def compute_mean(liste):
    """Calcule la moyenne d'une liste.
    @param liste: liste de nombres.
    @return: la moyenne de la liste.
    """
    total = 0
    for element in liste:
        total += element
    return total / len(liste)

def calculate_median_low(liste):
    """Calcule la médiane basse d'une liste.
    Args:
        liste: liste de nombres.
    Returns:
        la médiane de la liste.
    """
    liste = sorted(liste)
    return liste[len(liste) // 2]

# ... calcul de la moyenne et de la somme dans un 'if __name__ = "__main__":'
```

Exemple 2:

4 mauvaises pratiques à trouver

Objectif:

calculer la moyenne et la somme d'une liste

Retour Exemple 2

Lisibilité

Cohérence

En théorie :

- Un projet ne doit contenir qu'un seul format de code, standard de nommage, méthode de documentation et langue.
- Par défaut, suivre le standard officiel du langage.

En pratique :

- Nommer et documenter le code de manière consistante (une seule langue, une seule façon d'écrire la doc). Le plus important est d'en utiliser qu'une seule dans tout votre projet.
- Utiliser un linter de code pour analyser et formater le code : ruff

Retour Exemple 2

Lisibilité

DRY: Don’t Repeat Yourself

En théorie :

- Andy Hunt et Dave Thomas dans leur livre *The Pragmatic Programmer* : ***« Dans un système, toute connaissance doit avoir une représentation unique, non ambiguë, faisant autorité »***.

En pratique :

- Utiliser des classes, fonctions et librairies pour vous éviter d’écrire plusieurs fois le même code.
- ***NE PAS COPIER-COLLER***
  → importer garanti que vous utilisez un code testé dans lequel, si vous corrigez une erreur, tous vos programmes en bénéficient.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Lisibilité

YAGNI: You Ain’t Gonna Need It

En théorie :

- Minimiser le temps CPU et le temps mental des développeur.euse.s.
- N’écrire que ce dont on a besoin.
- Ne pas anticiper des besoins futurs non formalisés.

En pratique :

- Écrire le moins de code possible pour se concentrer sur l’essentiel.
- Tout code écrit devra être relu puis un jour maintenu, alors autant se simplifier la tâche.
- Écrire une abstraction seulement si vous avez déjà écrit 2 classes qui partagent les mêmes responsabilités.

Ne pas réinventer la roue

En théorie :

- Toute ligne de code écrite a le potentiel d'introduire un bug, mais plus un bout de code est utilisé, moins il a de chance d'être problématique.
  → Il est donc bénéfique de se reposer sur du code existant déjà testé par d'autres utilisateurs.

En pratique :

- Avant de coder une fonctionnalité, rechercher une librairie existante ne répond pas déjà à votre besoin. Le code y sera probablement de qualité, testé et maintenu.
  → pypi.org
- Sinon, alors chercher sur internet s'il existe du code open source que vous pouvez copier (en créditant l'auteur originel)

Correction Exemple 2:

cohérence dans la documentation

mean_and_sum.py U

mean_and_sum.py > ...

```python
def compute_sum(liste):
    """Calcule la somme d'une liste.
    Args:
        liste: liste de nombres.
    Returns:
        la somme de la liste.
    """
    total = 0
    for element in liste:
        total += element
    return total

def compute_mean(liste):
    """Calcule la moyenne d'une liste.
    @param liste: liste de nombres.
    @return: la moyenne de la liste.
    """
    total = 0
    for element in liste:
        total += element
    return total / len(liste)

def calculate_median_low(liste):
    """Calcule la médiane basse d'une liste.
    Args:
        liste: liste de nombres.
    Returns:
        la médiane de la liste.
    """
    liste = sorted(liste)
    return liste[len(liste) // 2]

# ... calcul de la moyenne et de la somme dans un 'if __name__ = "__main__":'
```

mean_and_sum.py U

mean_and_sum.py > ...

```python
from statistics import mean

if __name__ == "__main__":
    data = [12, 15, 18, 11]
    print(f"Mean: {mean(data)}")
    print(f"Sum: {sum(data)}")

```

main / monorepo4ai / projects / grokking

grokking

+ Find file Code



Grokking fix docker file
DEWASMES OSCAR authored 10 months ago and TOURNIER THEO committed 10 months ago

f9ea710e History

Name	Last commit	Last update
..		
grokking_v1	Grokking fix docker file	10 months ago
grokking_v2	Grokking clean project	2 years ago
grokking_v3(final)	Grokking fix docker file	10 months ago
image.png	Grokking fix docker file	10 months ago
README.md	grokking_doc	2 years ago
notes_soutenance.txt	Grokking fix docker file	10 months ago

README.md

Grokking

Contexte

Fait au labia en 2022.

Exemple 3:

4 mauvaises pratiques à trouver

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Documentation

README

En théorie :

- le README doit décrire le projet et surtout donner envie de l'utiliser.
- c'est un document synthétique (il peut pointer vers d'autres documentations si c'est un gros projet).

En pratique :

- Rédiger :
  - ce que fait le projet
  - comment l'installer et comment le lancer
  - détailler l'architecture du projet
  - donner les règles de contributions
  - ajouter les résultats d'outils automatiques : badges GitHub...
- Un grand README vaut mieux que pas de README !

Versionnage

git est un **outil de gestion de version** décentralisé, il permet de :

- Suivre les modifications du code dans le temps.
- Revenir facilement en arrière.
- Poser des “tags” pour enregistrer spécifiquement un état du code (version X.Y)

gitlab ou GitHub permettent de :

- Développer à plusieurs et de fusionner le travail avec les branches.
- Distribuer le code vers plusieurs dépôts distants (les “remote”).

Exemple : exploration du dépôt GitHub mfai.

Comment limiter les erreurs actuelles et à venir ?

Pour le moment, notre projet est :

- Documenté (README, documents techniques, schémas)
- Lisible (cohérent, simple, faible complexité cognitive)
- Versionné (git)
- Partagé (gitlab)
- .....
- .....
- .....

Collaboration

En théorie :

- De la même manière que dans la méthode scientifique, la relecture par les pairs permet de **garantir la qualité du code ajouté à un projet**.

En pratique :

1. Ajout d'une nouvelle fonctionnalité (et une seule !) dans une branche.
2. Créer une requête de fusion (Merge Request ou "MR") sur l'interface gitlab.
3. Accompagner cette MR d'un texte qui décrit ce qui a été modifié dans le code et pourquoi.
4. Les collègues lisent le code et proposent éventuellement des changements.
5. Quand tous les commentaires sont résolus, on fusionne la branche dans *main*.

Exemple : exploration d'une MR sur dépôt GitHub mfai.

Tests unitaires

En théorie :

- **Chaque fonction ou classe du code doit avoir un test unitaire associé**
- Les tests unitaires **garantissent la qualité fonctionnelle du projet**.
- Les tests unitaires **garantissent la non-régression du projet**.

En pratique :

- À l'ajout de fonctionnalité au code, écrire un nouveau test unitaire.
- À la mise à jour d'une fonctionnalité, mettre à jour le test unitaire associé.
- Utiliser pytest pour lancer les tests unitaires

docs.pytest.org/en/stable/

Exemple: exploration d'un test unitaire sur dépôt GitHub mfai.

Validation Automatique (ou CI)

En théorie :

- Toutes les modifications de code fusionnées à la branche main doivent avoir été validés.
- La branche *main* doit **toujours être une version fonctionnelle** du programme.

En pratique :

- gitlab et GitHub permettent d'exécuter du code avant chaque merge request grâce à l'intégration continue (CI pipelines).
- Ajouter les tests unitaires à la CI empêche de fusionner une branche qui ne valide pas tous les tests unitaires.
- Il est possible d'ajouter des outils de linting (ruff) ou de type checking (pyright) à la CI pour empêcher la fusion de code mal formaté

Reproductibilité de l'environnement

En théorie :

- À la manière de la méthode scientifique, le comportement d'un projet doit être reproductible.

En pratique :

- Votre projet utilise une version de langage spécifique, ainsi que des librairies qui, elles aussi, ont une version spécifique. Lorsque vous testez votre projet, **vous garantissez le bon comportement du code avec ces versions spécifiques**. Il faut donc spécifier ces versions (requirements.txt, ou autre).

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Qu’est-ce qu’un “bon code” ?

En résumé, un projet est maintenable si le code est :

- Documenté (docstring + documents techniques, schémas)
- Lisible (formatage cohérent, simple, faible complexité cognitive)
- Versionné (git)
- Partagé (gitlab)
- Testé unitairement
- Relu par les pairs (MR)
- Fonctionne dans un environnement reproductible
- Testé automatiquement (CI)

Attention : toutes ces bonnes pratiques sont parfois contradictoires, il faut donc les appliquer tout en s’adaptant aux besoins du projet !

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Patterns

Design patterns

En théorie :

- D'après refactoring.guru, *« Design patterns are typical solutions to commonly occurring problems in software design. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code. »*

En pratique :

- Un développeur rencontre fréquemment les mêmes problèmes à résoudre. De très bonnes solutions (ou patterns) ont été éprouvées et documentées. → Se familiariser avec celles-ci pour identifier les situations où elles sont utiles.

Patterns

Anti patterns

En théorie :

- Un développeur rencontre fréquemment les mêmes problèmes à résoudre. Certaines solutions à ces problèmes ne sont valables dans aucun scénarios.

En pratique :

- Lire et se familiariser avec ces exemples de choses à ne pas faire : docs.quantifiedcode.com/python-anti-patterns/

```
11  try:
12      do_something()
13  except:
14      pass
```

```
17  d = {"first_name": "Alfred", "last_name":"Hitchcock"}
18  for key in d:
19      print(f"{key} = {d[key]}")
```

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Astuces en Python

Une liste non exhaustive de bonnes astuces en Python :

- Docstring
- Librairie standard
- Exception
- *with statement*
- *f-string*
- Compréhension de liste
- *type-hinting*

Astuces en Python

Docstrings

En théorie :

- Python encourage le fait que le code serve de documentation.
- Avoir la documentation au plus proche du code aide à la lecture.
- Les docstrings permettent à votre éditeur de code d'afficher la documentation du projet pendant que vous l'éditez.

En pratique :

- C'est aujourd'hui un standard attendu par les développeurs python peps.python.org/pep-0257

```
33  class Clip(nn.Module):
34      """
35      Implementation of CLIP (Contrastive Langage-Image Pre-training) model.
36      - Based on the original article from OpenAI:
37          https://arxiv.org/abs/2103.00020
38      """
```

Astuces en Python

Utiliser la librairie standard

En théorie :

- La librairie standard contient du code qui répond à des besoins qui apparaissent fréquemment dans le code.
- Librairie testée et approuvé par toute la communauté python.

En pratique :

- Se familiariser avec la librairie standard docs.python.org/3/library
  - *pathlib* pour les chemins
  - *datetime* pour représenter les dates
  - *warnings* pour envoyer des alertes à l'utilisateur
  - *csv* pour lire les csv
  - *argparse* pour lire les arguments en ligne de commande
  - *statistics* pour les statistiques

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Astuces en Python

Les exceptions

En théorie :

- Conçues pour gérer des erreurs ou problèmes lors de l'exécution du code.

En pratique :

- Permet de couvrir des erreurs différentes (ValueError, FileNotFoundError, ZeroDivisionError...).
- Bannir le *pass* le *Except* ! Il faut au minimum renvoyer un *warning* (lib standard *warnings*) ou un *log* (lib standard *logging*).

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Astuces en Python

Les exceptions

```
3   import warnings
4
5   def safe_divide(dividend, divisor):
6       try:
7           return dividend / divisor
8       except ZeroDivisionError:
9           return 0
10      except (NameError, TypeError, ValueError) as e:
11          warnings.warn(str(e), Warning)
12
13  safe_divide('1', 1)
```

Astuces en Python

Le *with* statement

En théorie :

- Simplifie la gestion des ressources (fichiers, connexions réseaux…).
- Ouvre et clos les ressources même si une erreur survient.
- Équivalent à un *try:* … *finally:* … mais en plus court (+ simple à maintenir)

En pratique :

- Toujours utiliser *with* lors de l'ouverture d'un fichier ou d'une connexion :

```
1   with open("example.txt", "r") as file:
2       contents = file.read()
3       print(contents)  # Print file content
4       file.write("Hello, Python with statement!")
```

Astuces en Python

f-string

En théorie :

- Insérer des variables ou expressions à l'intérieur de chaînes de caractères

En pratique :

- Permet de simplifier le code, on remplace les + ou les “”.format() par un f””

```
4    message = "f-string"
5    print(f"Un exemple de {message}")
```

Astuces en Python

Compréhension de liste

En théorie :

- Créer et de manipuler des listes de manière concise et lisible

En pratique :

- Évite de faire appel à des boucles for et des conditions

```
data: list[int] = [1, 2, 3]
squared_data: list[int] = []
for item in data:
    if item > 1:
        squared_data.append(item**2)

squared_data: list[int] = [item ** 2 for item in data if item > 1]
```

Astuces en Python

Le type hinting

En théorie :

- Ajoute le type des objets dans le code.
- Les indices de type **aident le lecteur à comprendre** le code et à votre éditeur de code à auto-compléter de manière plus efficace.
- Les indices de type permettent l'utilisation de vérificateurs de type statiques qui permettent d'identifier les bugs avant l'exécution.

En pratique :

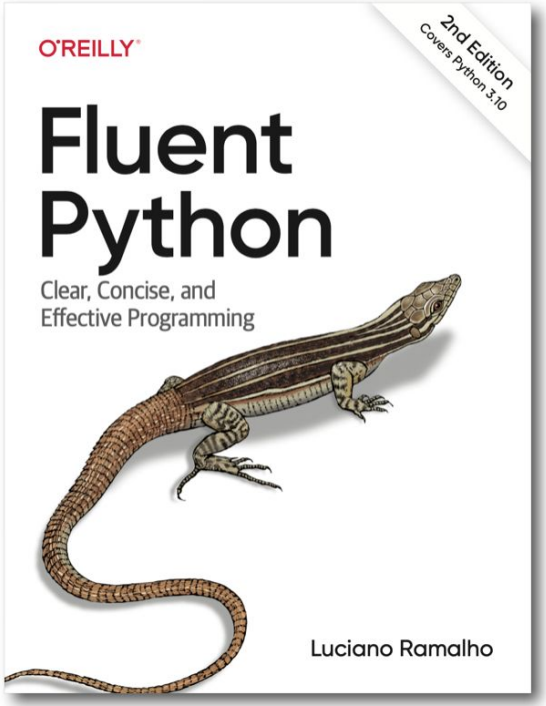
- Le type hinting est maintenant standard et attendu en python

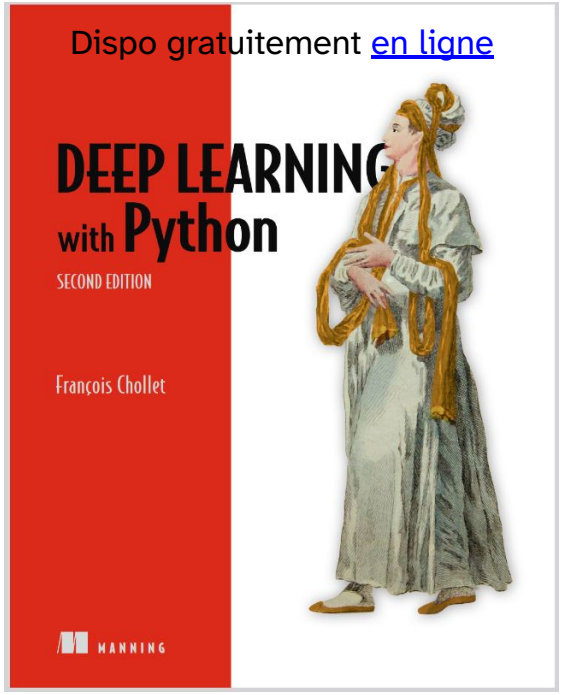
```
1    data: list[int] = [1, 2, 3]
2    value: int = data[0]
```

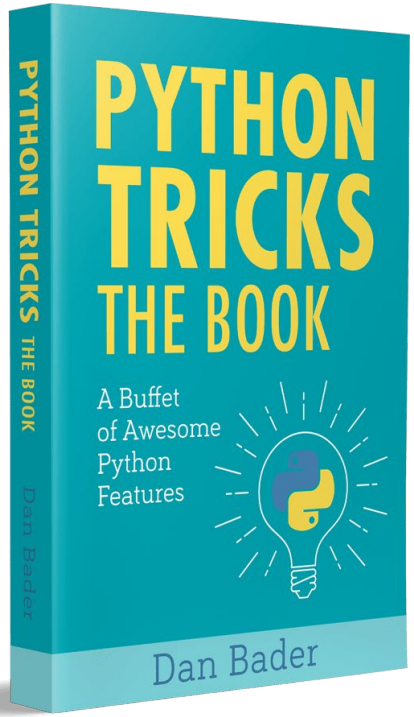
RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Pour aller plus loin…







Ou le site web fluentpython.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

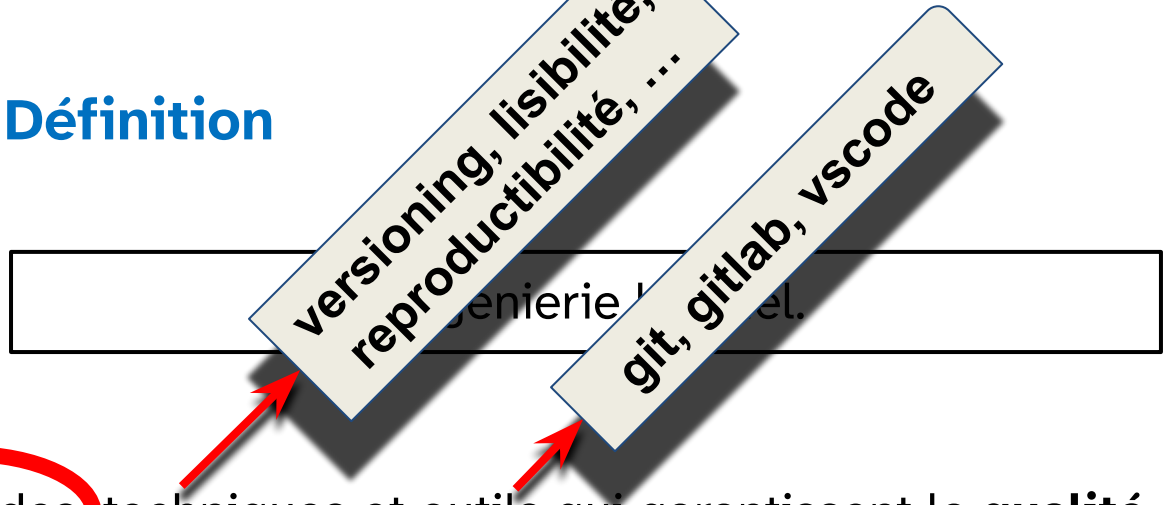
METEO FRANCE

Fin de la seconde partie

Troisième partie, sommaire

1. Importance du génie logiciel en entreprise
2. Cas particulier de Météo-France
3. La dette technique
4. Les paradigmes de développement
5. Les bonnes pratiques illustrées
6. **Le cycle de vie d'un projet logiciel**
7. **Les méthodologies de développement logiciel**

Définition



→ Méthodes, techniques et outils qui garantissent la **qualité** d'un logiciel.

→ Un logiciel est qualitatif s'il est **fonctionnel** et **maintenable**.

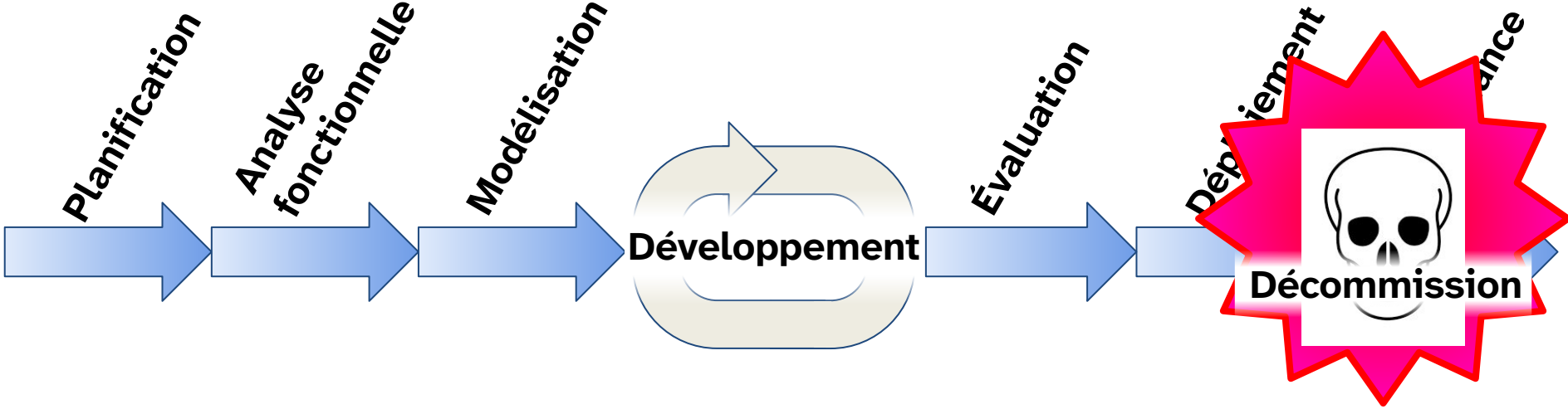
→ **Fonctionnel** = répond aux besoins du "client".
Maintenable = facile à mettre à jour par n'importe qui.

La méthodologie de développement logiciel



La méthodologie de réalisation d'un projet logiciel décrit comment gérer les différentes étapes de son ***cycle de vie****, comment réaliser ses actions et* ***livrables****.*

Le cycle de vie d'un projet logiciel



Planification

Client	DSM/LabIA
RH	1 élève IENM2, 6h
Quand	Cette semaine, en TP
Quoi	Évaluer le modèle de détection des sargasses
Livrable	Scores de précision, sur le dataset de validation. Plots pour comparer prédiction vs target.
Ressour--ces fournies	Modèle onnx, Dataset de validation (image input + masks target)
Outils	Python > matplotlib + numpy + onnxruntime
Étude de faisabilité.	

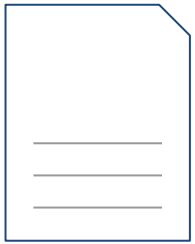
Définition fonctionnelle

Scores	- Moyenne sur tout le dataset de validation. - Matrice de confusion. - Précision. - Recall + Bonus: F1
Plots	- Sur quelques dates du dataset de validation. - Permet de comparer l'image OTCI, la prédiction du modèle et l'annotation du prévi.
Code	- Lisible. - Documenté. - Commenté. - Simple. *(maintenable)*

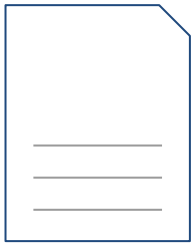
Modélisation

Data storage

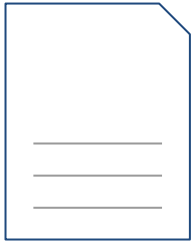
- sargasses/
  - sargasses_model.onnx
  - png/
    - YYYYMMDD.png
  - masks/
    - YYYYMMDD_<annotateur>.npz



compute_prediction.py –input_fo
- argparse
- load onnx module
- load input images
  - make prediction
  - save predictions

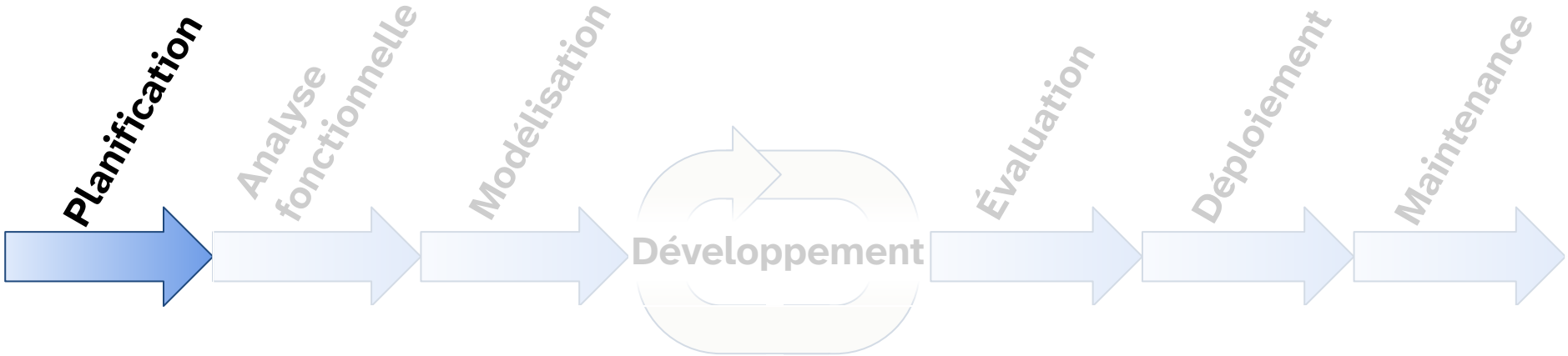


compute_scores.py –prediction_
- argparse
- load target and prediction
  - compute scores for of them
  - print mean scores



plot.py –input_file –target_file –p
- argparse
- load input, target and prediction
- plot them and save

Le cycle de vie d'un projet logiciel



Quel but avec quelles ressources ?

- But ?
- Ressources humaines ?
- Ressources du client ?
- Étude de faisabilité.
- Choix méthodologique.
- Livrables ?
- Dates du livrable ?

Détection des Sargasses par IA

Théo Tournier, Léa Berthomier, Frank Guibert & Bruno Pradel
DSM/LabIA

Journées R&D OM - 02/12/2024

Les enjeux

- **Algue brunes** pouvant former des bancs de **plusieurs km de long**
- **Libération de sulfure d'hydrogène** (H2S) et **ammoniaque** (NH3) lors de l'échouement
    - Sulfure d'hydrogène → odeur d'oeuf pourri → gêne des riverains et utilisateurs de la côte
    - Ammoniaque → impact sur la santé à long terme + détérioration prématurée des appareils électroménager
- **Demande un ramassage** pour ne pas impacter les écosystèmes du littoral → difficile en zone de mangrove (60% du littoral)
- **Engendre des pertes économiques et touristique** (5M€ estimé le 1er trimestre de 2015)

Plus d'informations sur les sargasses

Les enjeux



Les objectifs

Objectif: détection de sargasses à partir d'images satellitaires en utilisant des méthodes d'Intelligence Artificielle.
Le masque prévu alimente ensuite MOTHY (modèle de dérive) pour prévenir du risque d'échouage en Antilles-Guyane.

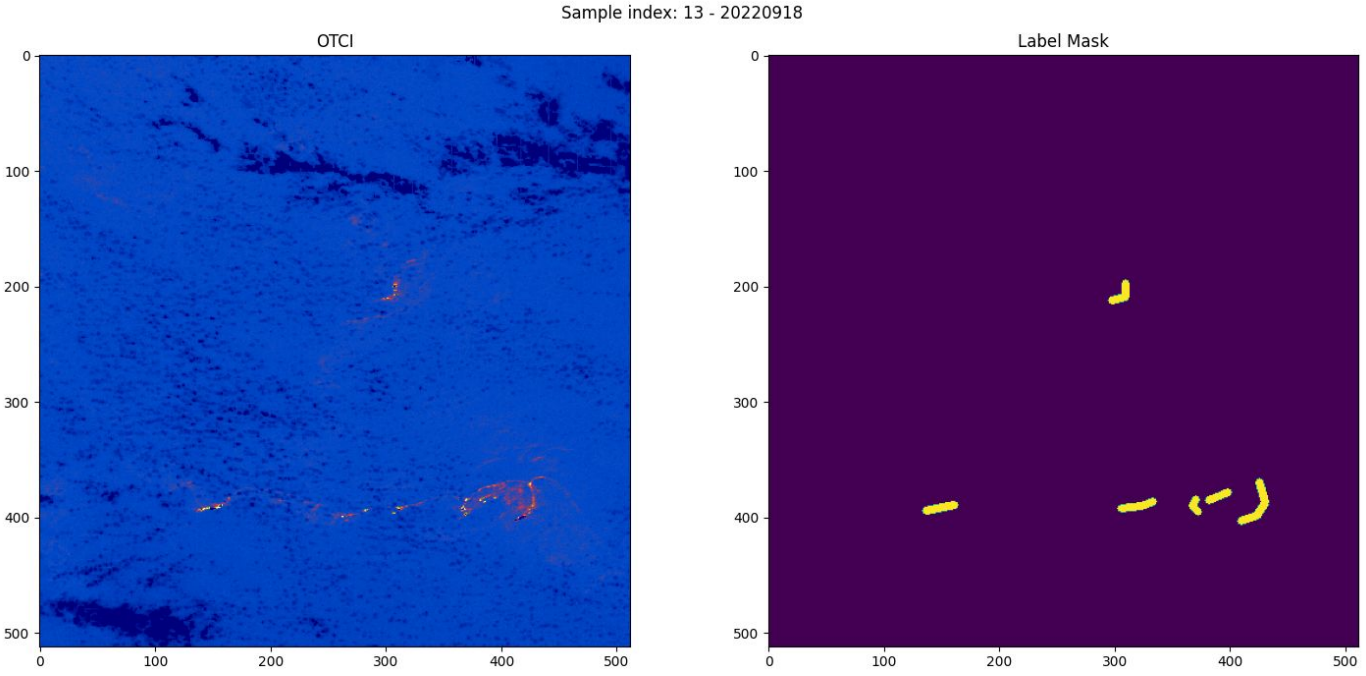
Demandeurs: DESR/CNRM/CEMS/Ocean

Produit actuel: analyse des bandes spectrales proche de l'IR sur des images pré-traitées. Les seuils de détection sont empiriques. Après détection, un post-processing est nécessaire pour éliminer les fausses détections.

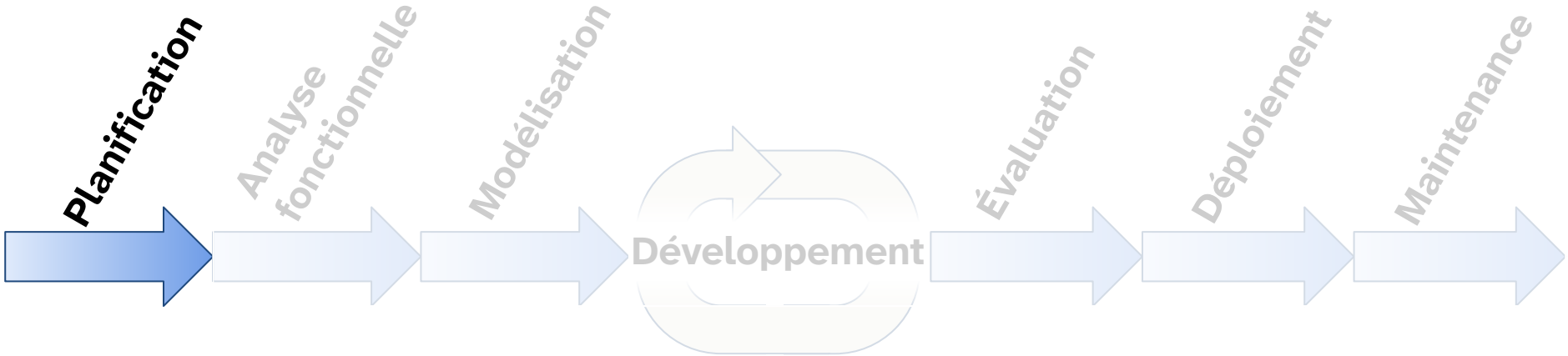
Jeu de données

Image OTCI

Masque



Le cycle de vie d'un projet logiciel



Quel but avec quelles ressources ?

- But ?
- Ressources humaines ?
- Ressources du client ?
- Étude de faisabilité.
- Choix méthodologique.
- Livrables ?
- Dates du livrable ?
- Méthode d'évaluation du livrable ?

Informations nécessaires à la rédaction d'un devis.

Le cycle de vie d'un projet logiciel



Quels sont précisément les besoins du client ?

Dialogue avec le client pour définir ses besoins.
Définition des **livrables** et leur **évaluation**.

Informations nécessaires à la rédaction d'un cahier des charges.

Le cycle de vie d'un projet logiciel



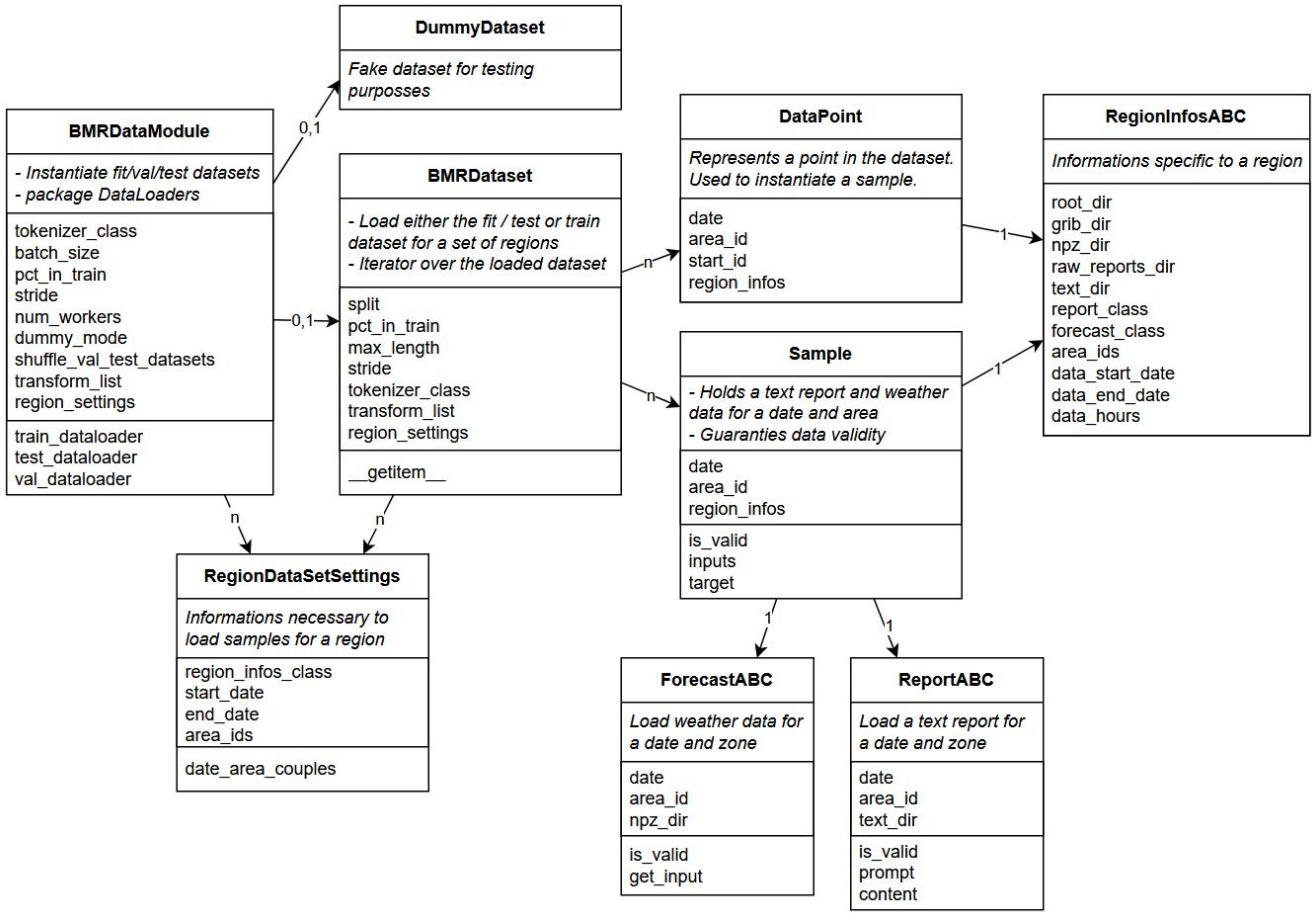
Comment ?

Définir avec l'équipe de développement l'architecture du logiciel, la structure du code, ses **interfaces** et le détail de son fonctionnement.

La modélisation

Des schémas pour représenter:

Les relations entre des donnés.

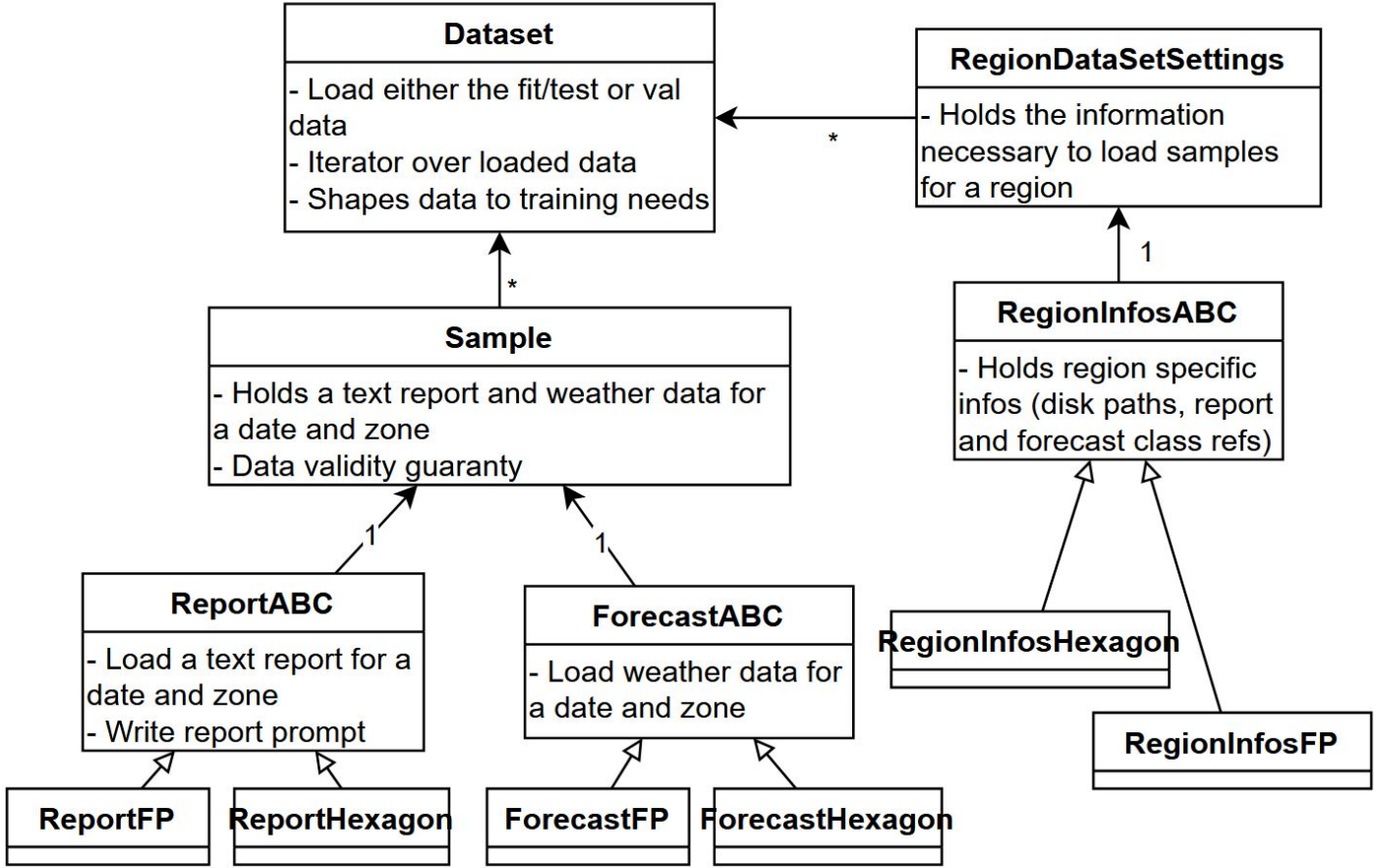


La modélisation

La modélisation

Des schémas pour représenter:

Les relations et les responsabilités des entités constituant un projet.



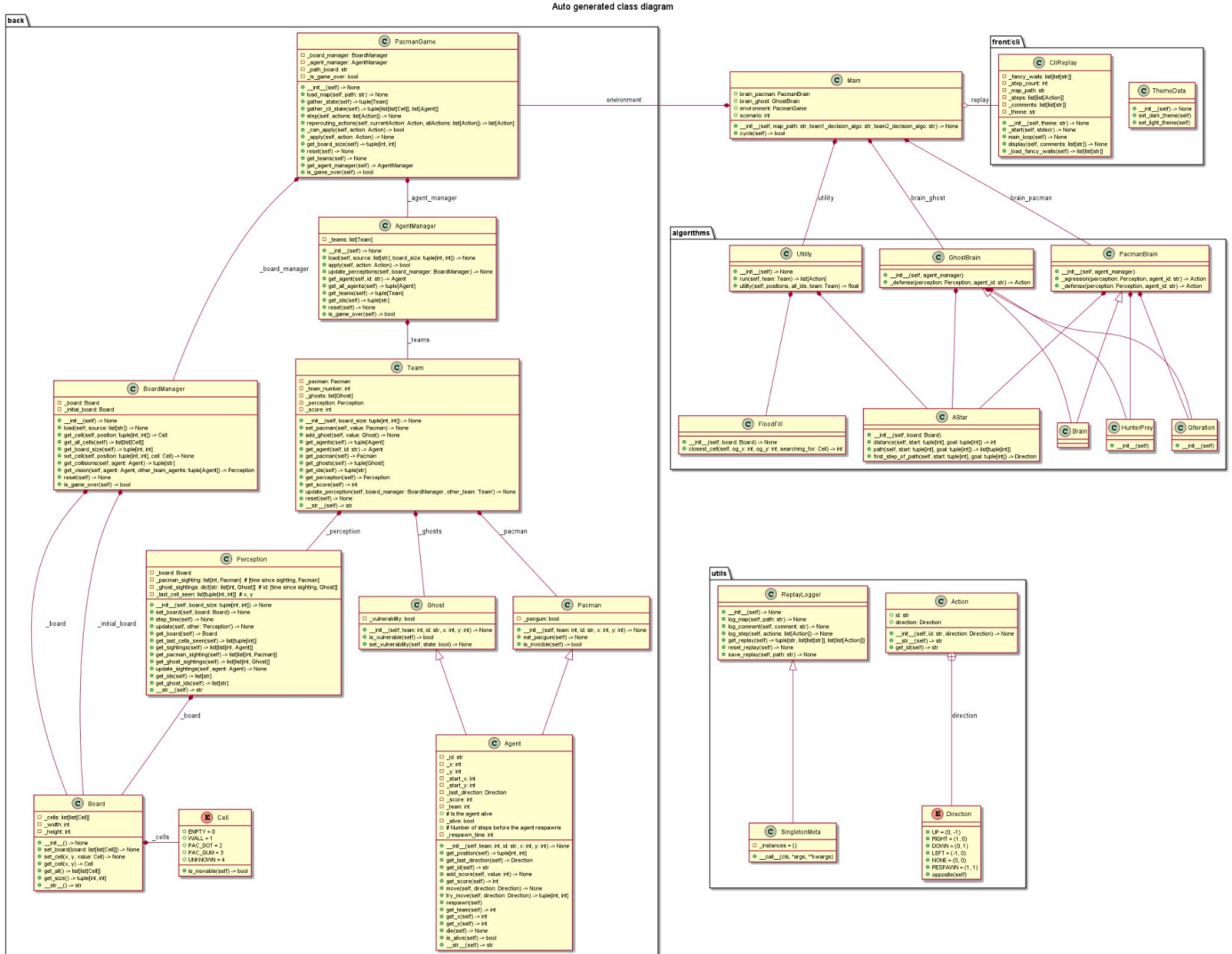
RÉPUBLIQUE FRANÇAISE
Liberté Égalité Fraternité

METEO FRANCE

La modélisation

Des schémas pour représenter:

Les interfaces entre les classes d'un programme orienté objet.

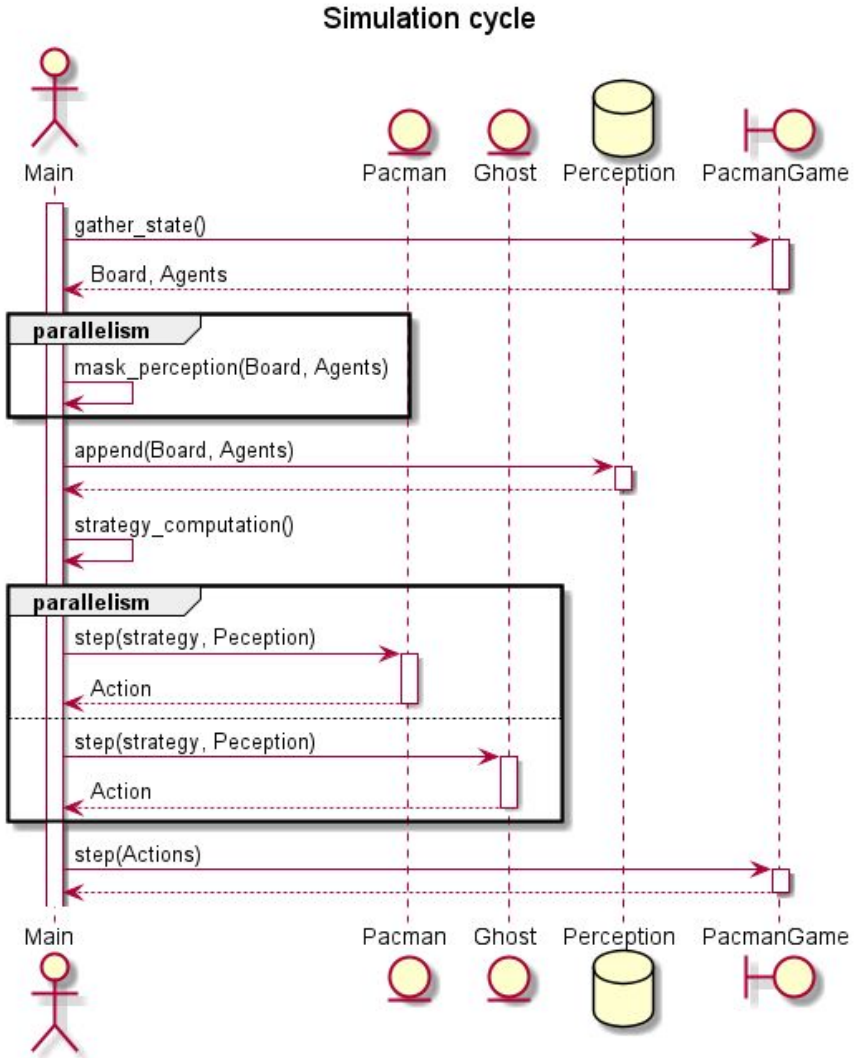


RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

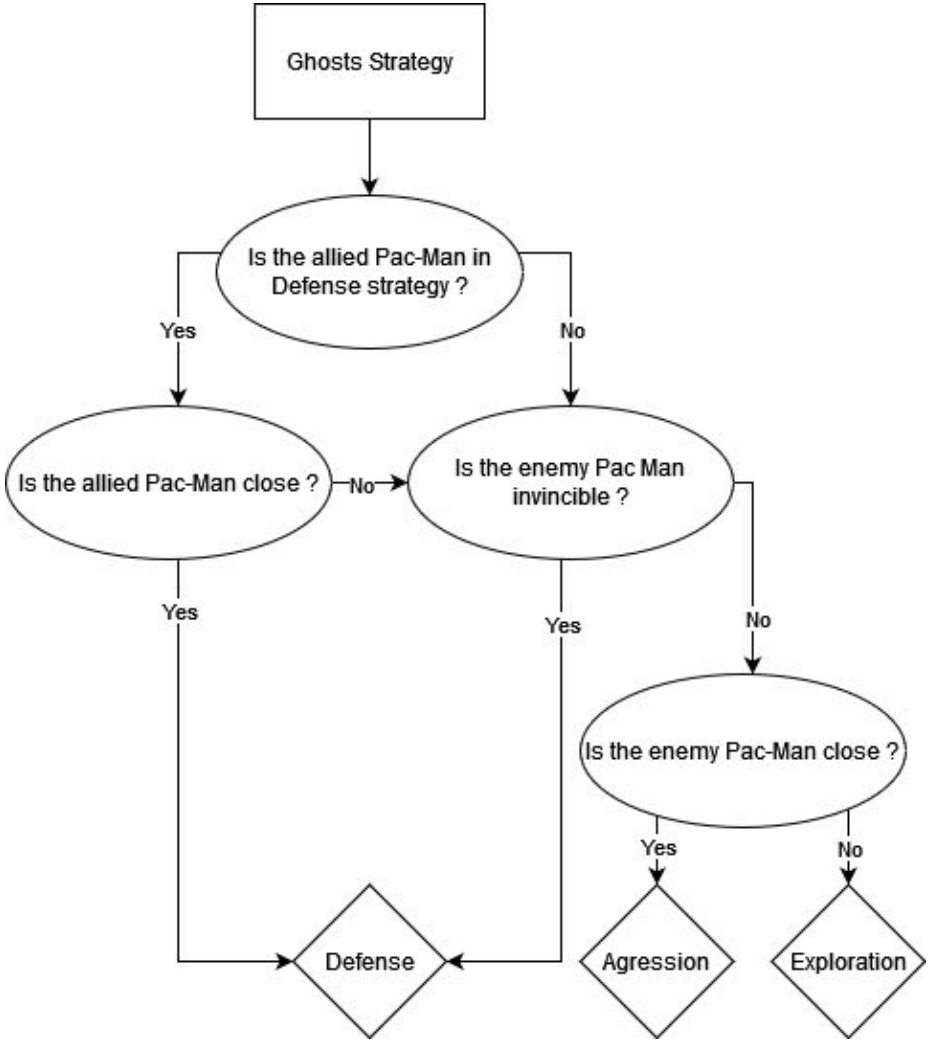
Des schémas pour représenter:

Les séquences d'actions à l'intérieur d'un programme.



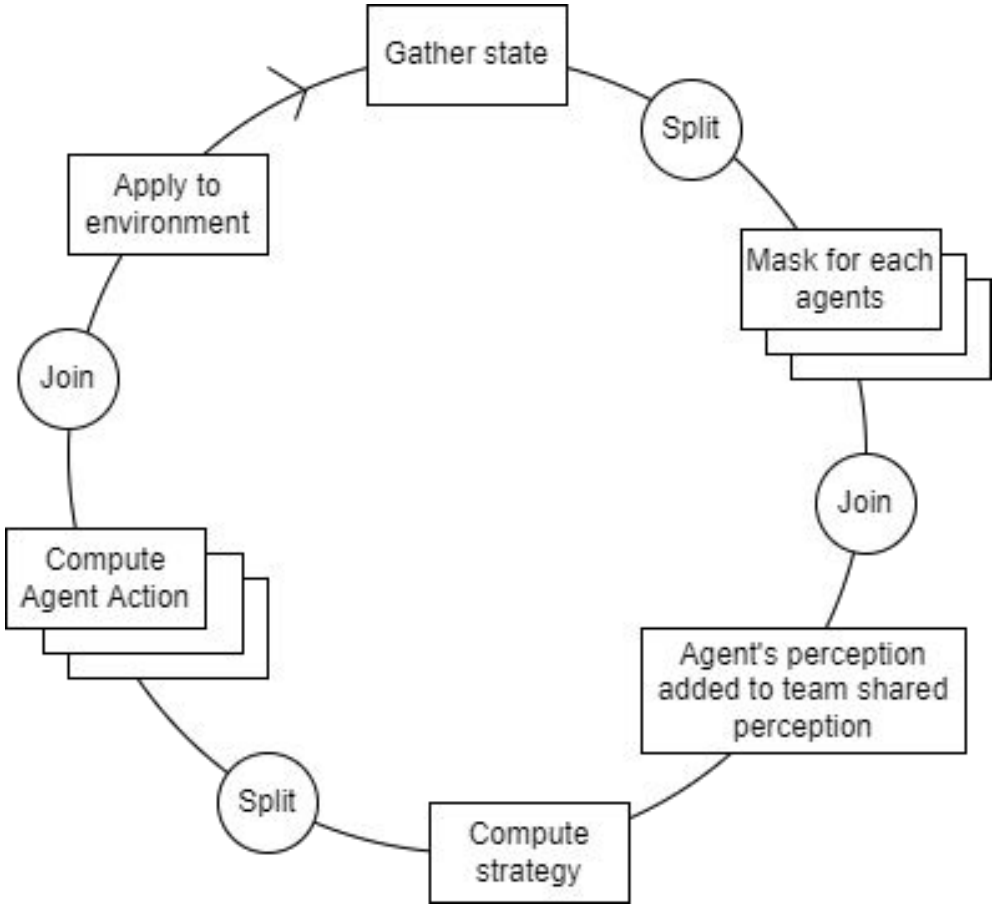
Des schémas pour représenter:

Un schéma de décision.



Des schémas pour représenter:

Le comportement cyclique d'un programme.



Des schémas pour représenter:

Une hiérarchie de dossiers.

```
scratch/shared/bmr/data/
├── polynesia
│   ├── grib
│   │   └── YYYY
│   │       └── YYYY_MM_DD_HH_model.grib
│   ├── npz
│   │   └── YYYY
│   │       └── YYYY_MM_DD_HH_zoneid_model.npz
│   ├── raw_reports
│   │   ├── YYYY
│   │   │   └── YYYY_MM_DD_HH.(pdf|DOC)
│   │   ├── extracted_docs
│   │   │   └── YYYY
│   │   │       └── YYYY_MM_DD_HH.txt  # text versions of the .DOC raw reports
│   │   └── unknown_conversion_dict.txt  # dict used to fix bad conversions
│   └── text
│       └── YYYY
│           └── YYYY_MM_DD_HH_zoneid.txt
└── hexagon
    ├── grib
    │   └── YYYYMMDDHH_model.grib
```

Le cycle de vie d'un projet logiciel

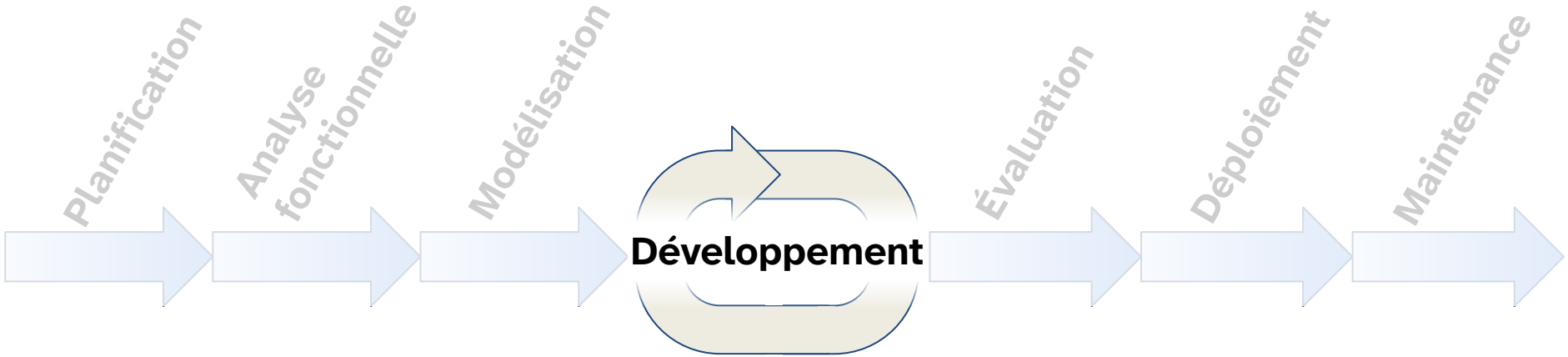


Comment ?

Définir avec l'équipe de développement l'architecture du logiciel, la structure du code, le détail de son fonctionnement.

Informations nécessaires à la rédaction du code.

Le cycle de vie d'un projet logiciel



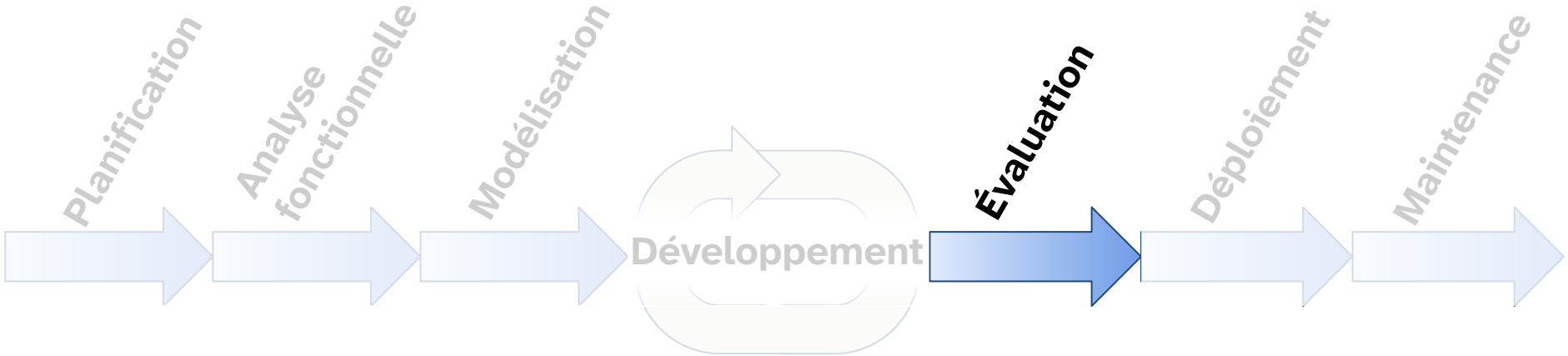
Écrire le code

Respecter les bonnes pratiques.

Appliquer un plan de mitigation de la dette technique.

Le code est maintenable, et facile à mettre à jour si le besoin client évolue.

Le cycle de vie d'un projet logiciel



Comment on sait que le projet est fini ?

Application de l'évaluation définie avec le client dans le cahier des charges.
Si l'évaluation est positive, on passe au déploiement.

Le cycle de vie d’un projet logiciel



Mise en opérationnelle.

Application de la méthode de déploiement définie avec le client dans le cahier des charges.

Le cycle de vie d’un projet logiciel



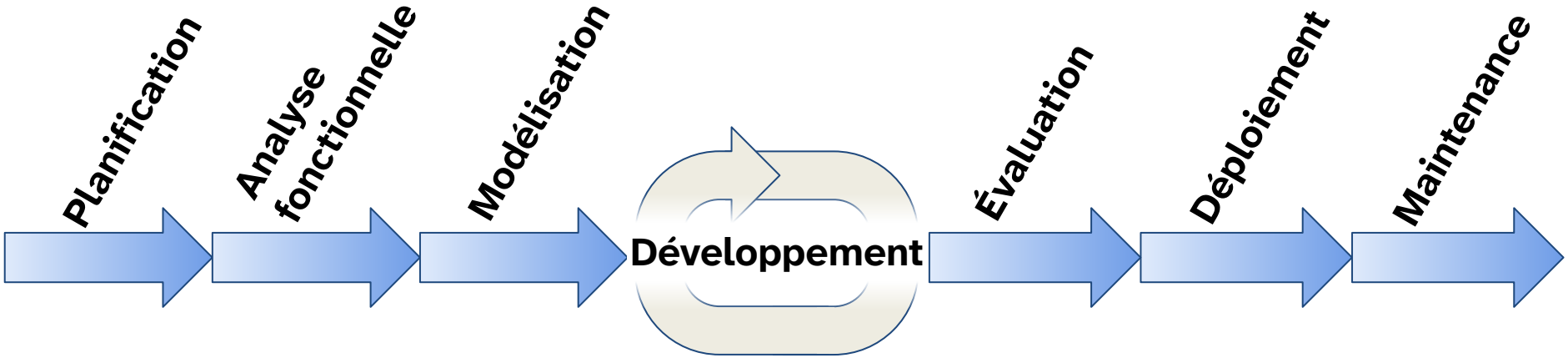
Maintien en opérationnel.

Application de la méthode de déploiement définie avec le client dans le cahier des charges.

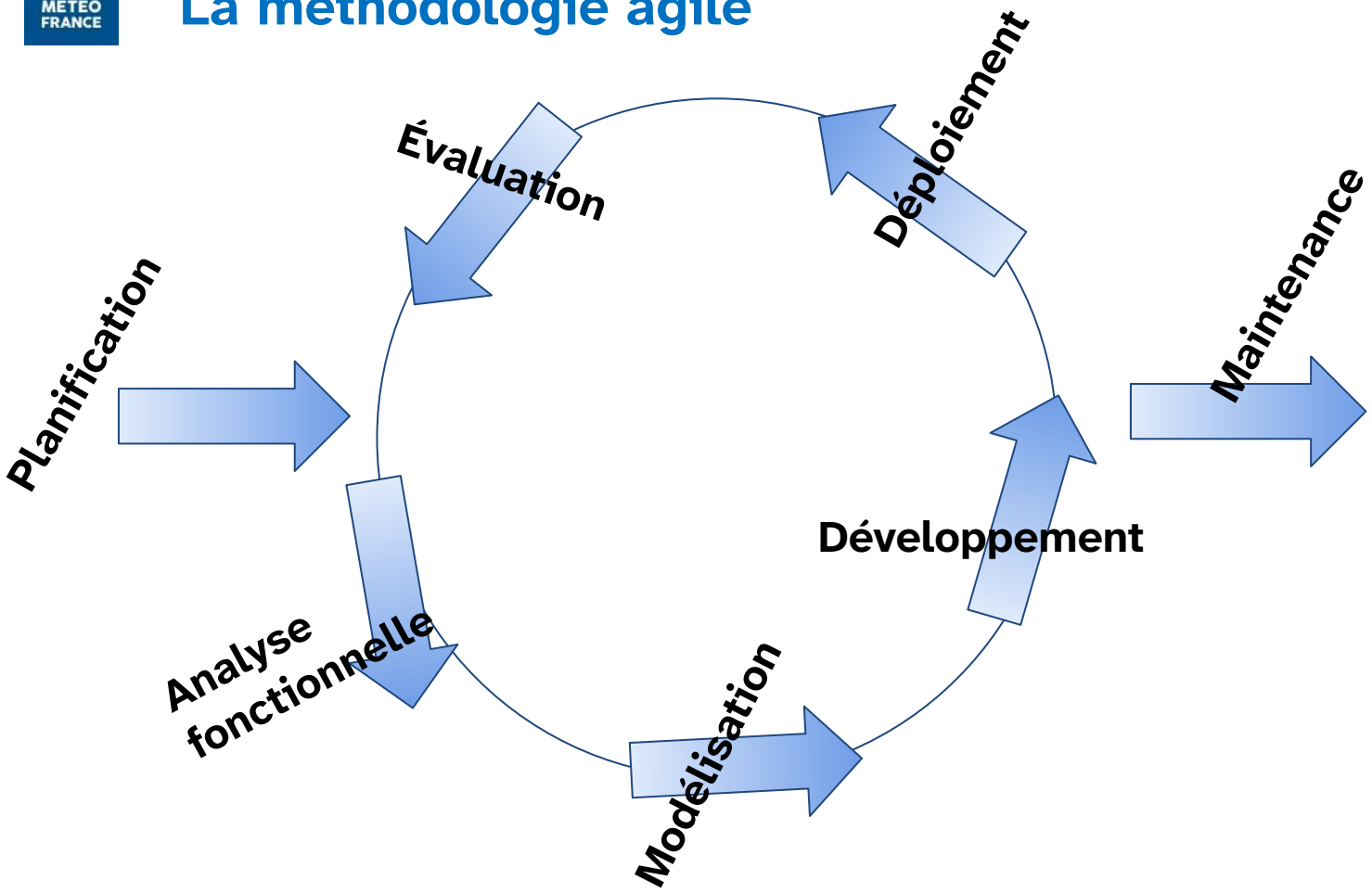
La méthodologie de développement logiciel

La méthodologie de réalisation d'un projet logiciel décrit ***comment gérer les différentes étapes de son cycle de vie,*** *comment réaliser ses actions et livrables.*

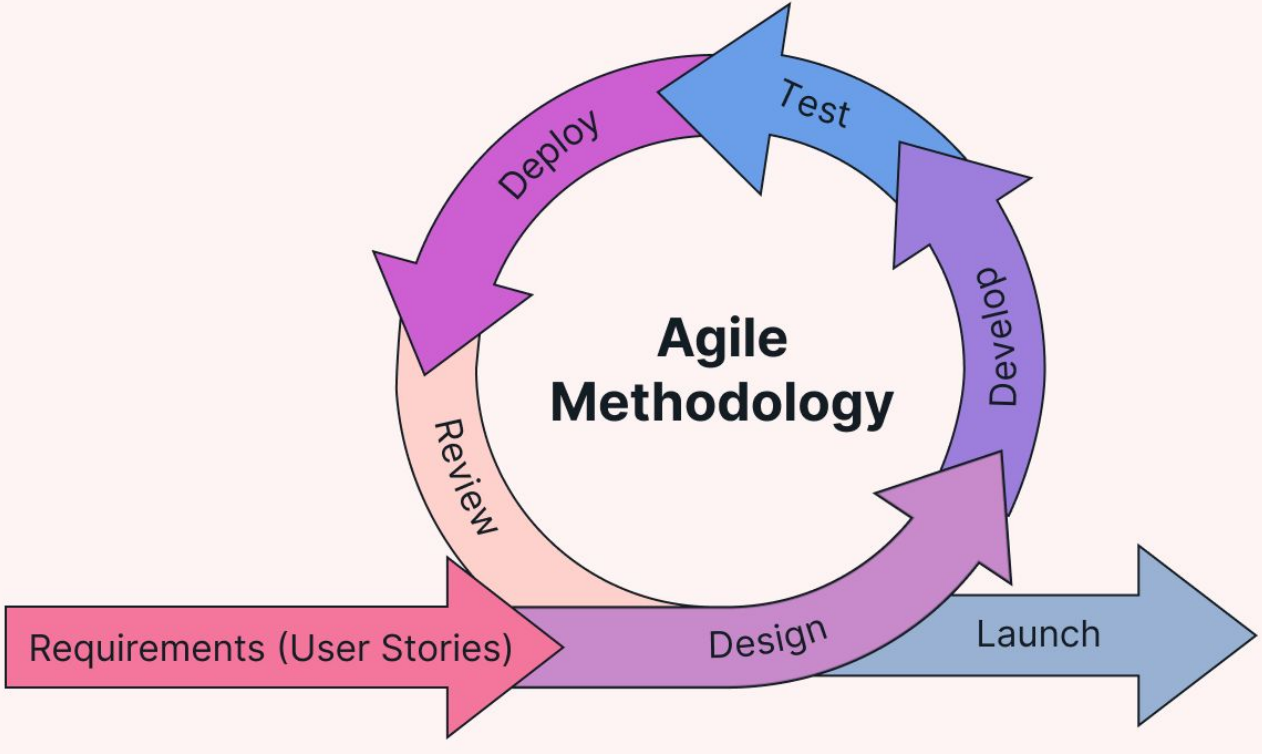
La méthodologie agile



La méthodologie agile

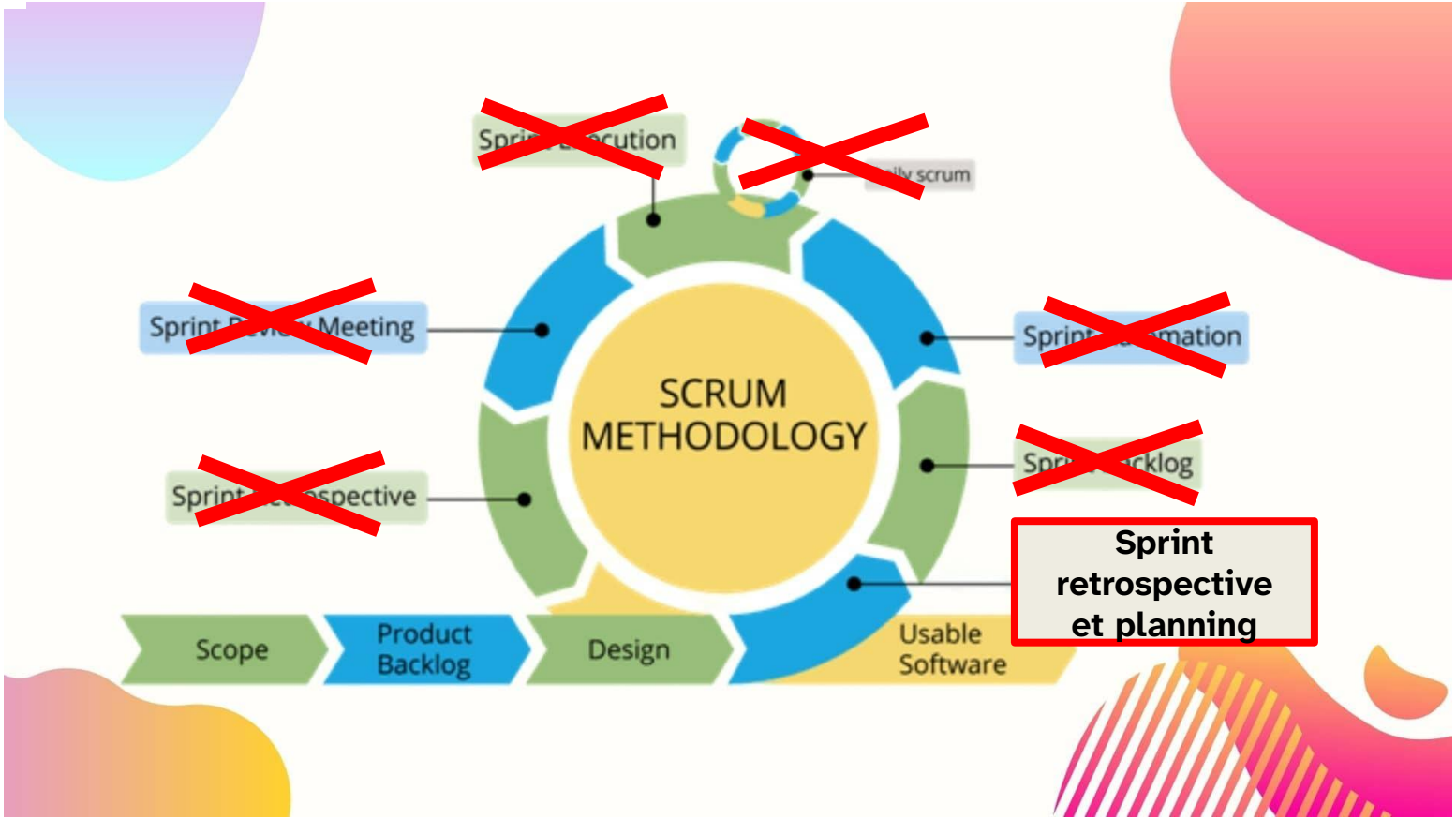


La méthodologie Agile



@https://www.usemotion.com/blog/agile-methodology

La méthodologie Agile: Scrum



@https://www.programmingwithbasics.com/2020/09/scrum-and-agile.html

La méthodologie Agile: Kanban



@https://www.savio.io/product-roadmap/kanban-roadmap/

La méthodologie Agile: Au LabIA



RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Fin du cours, résumé

1. **Importance du génie logiciel en entreprise**
2. **Cas particulier de Météo-France**
3. **La dette technique**
4. **Les paradigmes de développement**
5. **Les bonnes pratiques illustrées**
6. **Le cycle de vie d’un projet logiciel**
7. **Les méthodologies de développement logiciel**

Bonus: ADR (Architectural Decision Record)

Un projet est fait d'une série de décisions, certaines annulant des décisions précédentes... C'est très utile d'avoir un document (à la fin du readme ?) qui liste toutes les décisions importantes du projet, avec sa date et raison.

EX:

- 01/04/2023: Change le langage de programmation de Fortran à python suite à une évolution des ressources humaines disponibles pour le projet.
- 02/06/2023: Stockage du dataset en netcdf à la place du format grib pour une meilleure compression et vitesse de chargement en mémoire.
- 12/08/2023: Change l'environnement de développement de Docker à Singularity pour garantir la stabilité du projet en opérationnel.

Nota Bene: IA génératives d'aide au développement

- Demandez à vos chef.fe.s si c'est permis et si oui quelle technologie est autorisée, si besoin il/elle escaladera vers la DSI (enjeux de cybersécurité et protection des données de l' établissement)
- Gain de productivité de 30 à 40 % selon les études
- Permet aussi de se faire expliquer du code "compliqué"
- github copilot ou chatGPT ou des solutions déployées localement (demandez à la DSI)

Travaille en autonomie

Explorer ces différentes newsletters et blog techniques, prenez votre temps et souscrivez avec votre email @meteo.fr ou un mail perso si vous voulez que cela vous suive “hors MF”

- https://pythonspeed.com/
- https://realpython.com/
- https://pycoders.com/
- https://www.python.org/psf/newsletter/
- https://alphasignal.ai/
- https://dataelixir.com

Si vous ne vous sentez pas l’âme d’un.e Pythoniste, cherchez une newsletter scientifique.

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

METEO FRANCE

Travaille en autonomie

Internet regorges de ressources python et géni logiciel, vidéos sur YouTube, cheat sheets, blogs,et des forums avec des gens qui se posent les mêmes questions que vous.

- ⭐ Un jeu dans le navigateur web pour apprendre git.
- ⭐ Une liste exhaustive de choses à ne pas faire en python.
- ⭐ Un site web interactif pour construire un réseau de neurones.
- ⭐ Vidéo de *3 blue 1 brown* qui explique le deep learning.
- La cheat sheet git la plus complète possible
- Les 10 commandements de la programation sans ego.
- Vidéo introduction par *Fireship* au concepte de Monorepo utilisé au LabIA.
- All Machine Learning algorithms explained in 17 min par *Infinite Codes.*